SQL · MYSQL · DATABASE DESIGN & ANALYTICS

Banking Management System

An end-to-end relational database project: 18 normalised tables, seed data, joins, subqueries, CTEs, window functions, views, stored routines, triggers and recursive queries.

18	59	120+	12
TABLES	SQL STEPS	RESULT SETS	SQL TOPICS

Case study documentation · SQL scripts, schema, queries and results

Contents

Part I — Schema Design 7

1 — Create Database 7
2 — Create Branches Table 7
3 — Create Customers Table 7
4 — Create AccountTypes Table 8
5 — Create Accounts Table 8
6 — Create Transactions Table 8
7 — Create Beneficiaries Table 9
8 — Create FundTransfers Table 9
9 — Create Loans Table 9
10 — Create LoanPayments Table 10
11 — Create Cards Table 10
12 — Create FixedDeposits Table 10
13 — Create RecurringDeposits Table 11
14 — Create ATMTransactions Table 11
15 — Create KYC Table 11
16 — Create Employees Table 12
17 — Create Complaints Table 12
18 — Create InternetBanking Table 12
19 — Create AuditLogs Table 13

Part II — Data Loading & Verification 14

20 — Insert Account Types Data 14
21 — Insert Branches Data 14
22 — Insert Employees Data 15
23 — Insert Customers Data 15
24 — Insert Accounts Data 16
25 — Insert Transactions Data 16
26 — Insert Beneficiaries Data 16
27 — Create FundTransfers Table 17
28 — Insert Loans Data 17
29 — Insert Loan Payments Data 17
30 — Insert Cards Data 18
31 — Insert Fixed Deposits Data 18
32 — Insert Recurring Deposits Data 18

33 — Create ATMTransactions Table . . . 19
34 — Insert KYC Data . . . 19
35 — Insert Complaints Data . . . 19
36 — Insert Internet Banking Data . . . 20
37 — Insert Audit Logs Data . . . 20
38 — Verify All Tables . . . 20

Part III — Querying & Analysis . . . 22

39 — Basic SELECT Queries . . . 22
40 — Filtering with Operators . . . 24
41 — SQL Functions . . . 28
42 — GROUP BY and HAVING . . . 31
43 — DISTINCT and NULL Handling . . . 33
44 — Inner Join . . . 36
45 — Left Join . . . 40
46 — Right Join . . . 43
47 — Self Join . . . 46
48 — Multi-Table JOIN . . . 48
48B — Insert Fund Transfer Data . . . 50
49 — Subqueries . . . 55
50 — Common Table Expressions (CTE) . . . 59
51 — Set Operations . . . 63
52 — Window Functions . . . 68

Part IV — Advanced SQL Objects . . . 72

53 — Views . . . 72
54 — Indexes . . . 77
55 — Stored Procedures . . . 79
56 — Stored Procedure Execution . . . 82
57 — Stored Functions . . . 83
58 — Triggers . . . 86
59 — Recursive CTE . . . 92

Project Overview

The **Banking Management System** is a SQL-based database project that organises the core operations of a bank: branches, customers, accounts, transactions, fund transfers, loans, cards, fixed and recurring deposits, KYC, complaints, employees, internet banking and audit logs. It starts from schema design with keys and constraints and builds up to analytical SQL, reusable views, stored routines, triggers and recursive queries.

18	59	10	13	19,300,000
TABLES	SQL STEPS	CUSTOMERS	TRANSACTIONS	LOANS ISSUED

SQL TECHNIQUES DEMONSTRATED

Schema & Constraints
PRIMARY / FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT and CHECK rules

Filtering & Functions
WHERE operators, LIKE, BETWEEN, aggregates, string and numeric functions

GROUP BY & HAVING
Grouped summaries with post-aggregation filters

NULL Handling
IS NULL, COALESCE, DISTINCT

Joins
INNER, LEFT, RIGHT, SELF and multi-table joins

Subqueries & CTEs
Scalar, IN / EXISTS, ALL, correlated and WITH queries

Set Operations
UNION, UNION ALL, INTERSECT-style and EXCEPT-style queries

Window Functions
ROW_NUMBER, RANK, running totals, PARTITION BY

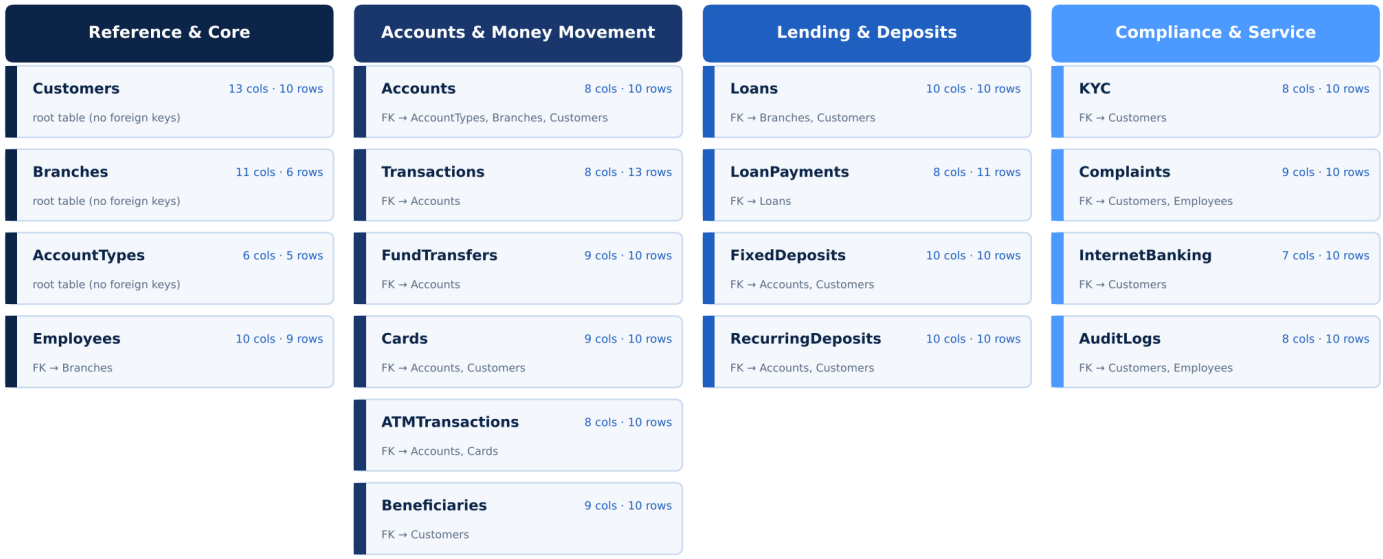
Views & Indexes
Reusable reporting views and performance indexes

Stored Routines
Procedures, functions and their execution

Triggers & Audit
Automatic audit logging on data changes

Recursive CTEs
Hierarchies, number series and date generation

DOMAIN MAP · 18 TABLES IN FOUR DOMAINS



Schema Summary

All 18 tables with their column counts, foreign-key relationships and seed rows loaded by the insert steps.

Table	Purpose	Cols	Foreign keys	Rows
Branches	Branch master with manager and status	11	—	6
Customers	Customer profile and contact details	13	—	10
AccountTypes	Savings / Current / Salary product rules	6	—	5
Accounts	Customer accounts and balances	8	AccountTypes, Branches, Customers	10
Transactions	Deposits, withdrawals, transfers	8	Accounts	13
Beneficiaries	Saved payees per customer	9	Customers	10
FundTransfers	NEFT / RTGS / IMPS / UPI transfers	9	Accounts	10
Loans	Loan products issued to customers	10	Branches, Customers	10
LoanPayments	EMI and repayment history	8	Loans	11
Cards	Debit and credit cards	9	Accounts, Customers	10
FixedDeposits	Term deposits and maturity	10	Accounts, Customers	10
RecurringDeposits	Monthly recurring deposit plans	10	Accounts, Customers	10
ATMTransactions	Card based ATM activity	8	Accounts, Cards	10
KYC	Identity verification records	8	Customers	10
Employees	Bank staff by branch	10	Branches	9
Complaints	Customer complaints and assignment	9	Customers, Employees	10
InternetBanking	Online banking credentials & status	7	Customers	10
AuditLogs	Action trail for key entities	8	Customers, Employees	10

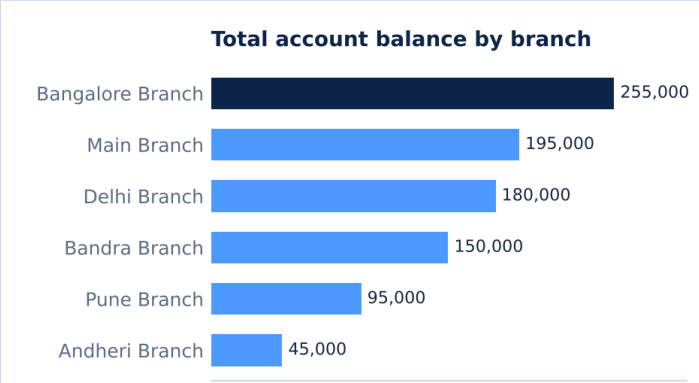
HOW TO READ THIS DOCUMENT

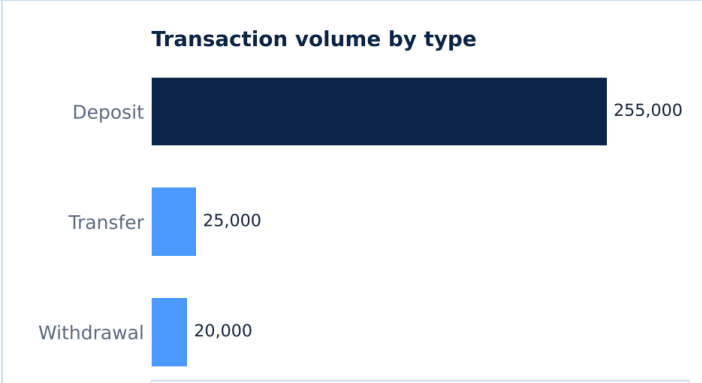
Each step shows a numbered banner, the original SQL with syntax highlighting and, for queries, the result table. Part I covers schema design, Part II loads the data, Part III covers querying and Part IV covers views, indexes, routines, triggers and recursive queries.

Data Snapshot & Key Findings

Headline figures from the seeded dataset, computed with the same aggregate queries used in Part III.

28%	**Bangalore Branch** holds the largest share of balances: 255,000 of 920,000 across 6 branches.
85%	**Deposits** dominate transaction volume: 255,000 of 300,000 over 13 transactions.
47%	**Business Loan** is the biggest loan category at 9,000,000 of a 19,300,000 portfolio.
18	**Eighteen related tables** linked by foreign keys keep customers, accounts, lending and compliance data consistent.









Charts use the seed data loaded in Part II. Figures are plain numeric amounts as stored in the tables.

PART I

Schema Design

Creating the BankingSystem database and its 18 related tables with primary keys, foreign keys, defaults and CHECK constraints.

18 tables	PK / FK	CHECK
Normalised design	Referential integrity	Domain rules

1 Create Database

PART I

```
CREATE DATABASE BankingSystem;

USE BankingSystem;
```

2 Create Branches Table

PART I

```
CREATE TABLE Branches (
    branch_id INT PRIMARY KEY AUTO_INCREMENT,
    branch_name VARCHAR(100) NOT NULL,
    branch_code VARCHAR(20) UNIQUE NOT NULL,
    address VARCHAR(255),
    city VARCHAR(50),
    state VARCHAR(50),
    pincode VARCHAR(10),
    phone VARCHAR(15),
    manager_name VARCHAR(100),
    opening_date DATE,
    status VARCHAR(20) DEFAULT 'Active'
);
```

3 Create Customers Table

PART I

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    phone VARCHAR(15) UNIQUE,
    date_of_birth DATE,
    gender VARCHAR(10),
    address VARCHAR(255),
    city VARCHAR(50),
    state VARCHAR(50),
    pincode VARCHAR(10),
    registration_date DATE DEFAULT (CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'Active'
);
```

4 Create AccountTypes Table PART I

```sql
CREATE TABLE AccountTypes (
    account_type_id INT PRIMARY KEY AUTO_INCREMENT,
    account_type_name VARCHAR(50) NOT NULL UNIQUE,
    minimum_balance DECIMAL(12,2) DEFAULT 0,
    interest_rate DECIMAL(5,2) DEFAULT 0,
    description VARCHAR(255),
    status VARCHAR(20) DEFAULT 'Active'
);
```

5 Create Accounts Table PART I

```sql
CREATE TABLE Accounts (
    account_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    branch_id INT NOT NULL,
    account_type_id INT NOT NULL,
    account_number VARCHAR(20) UNIQUE NOT NULL,
    balance DECIMAL(15,2) DEFAULT 0,
    opening_date DATE DEFAULT (CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id),
    FOREIGN KEY (account_type_id) REFERENCES AccountTypes(account_type_id),

    CHECK (balance >= 0)
);
```

6 Create Transactions Table PART I

```sql
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    transaction_type VARCHAR(20) NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    transaction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    description VARCHAR(255),
    reference_number VARCHAR(50) UNIQUE,
    status VARCHAR(20) DEFAULT 'Success',

    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),

    CHECK (amount > 0),
    CHECK (transaction_type IN ('Deposit', 'Withdrawal', 'Transfer'))
);
```

7 Create Beneficiaries Table PART I

```sql
CREATE TABLE Beneficiaries (
    beneficiary_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    beneficiary_name VARCHAR(100) NOT NULL,
    account_number VARCHAR(20) NOT NULL,
    bank_name VARCHAR(100),
    ifsc_code VARCHAR(20),
    nickname VARCHAR(50),
    added_date DATE DEFAULT (CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

8 Create FundTransfers Table PART I

```sql
CREATE TABLE FundTransfers (
    transfer_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_account_id INT NOT NULL,
    receiver_account_id INT NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    transfer_type VARCHAR(20) NOT NULL,
    transfer_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    reference_number VARCHAR(50) UNIQUE,
    remarks VARCHAR(255),
    status VARCHAR(20) DEFAULT 'Success',

    FOREIGN KEY (sender_account_id) REFERENCES Accounts(account_id),
    FOREIGN KEY (receiver_account_id) REFERENCES Accounts(account_id),

    CHECK (amount > 0),
    CHECK (transfer_type IN ('NEFT', 'RTGS', 'IMPS', 'UPI'))
);
```

9 Create Loans Table PART I

```sql
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    branch_id INT NOT NULL,
    loan_type VARCHAR(50) NOT NULL,
    loan_amount DECIMAL(15,2) NOT NULL,
    interest_rate DECIMAL(5,2) NOT NULL,
    tenure_months INT NOT NULL,
    start_date DATE,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id),

    CHECK (loan_amount > 0),
    CHECK (interest_rate >= 0),
    CHECK (tenure_months > 0)
);
```

10 Create LoanPayments Table PART I

```sql
CREATE TABLE LoanPayments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    loan_id INT NOT NULL,
    payment_amount DECIMAL(15,2) NOT NULL,
    payment_date DATE DEFAULT (CURRENT_DATE),
    payment_method VARCHAR(30),
    transaction_reference VARCHAR(50) UNIQUE,
    remarks VARCHAR(255),
    status VARCHAR(20) DEFAULT 'Paid',

    FOREIGN KEY (loan_id) REFERENCES Loans(loan_id),

    CHECK (payment_amount > 0)
);
```

11 Create Cards Table PART I

```sql
CREATE TABLE Cards (
    card_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    account_id INT NOT NULL,
    card_type VARCHAR(20) NOT NULL,
    card_number VARCHAR(20) UNIQUE NOT NULL,
    issue_date DATE DEFAULT (CURRENT_DATE),
    expiry_date DATE NOT NULL,
    card_status VARCHAR(20) DEFAULT 'Active',
    daily_limit DECIMAL(12,2) DEFAULT 50000,

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),

    CHECK (card_type IN ('Debit', 'Credit')),
    CHECK (daily_limit > 0)
);
```

12 Create FixedDeposits Table PART I

```sql
CREATE TABLE FixedDeposits (
    fd_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    account_id INT NOT NULL,
    principal_amount DECIMAL(15,2) NOT NULL,
    interest_rate DECIMAL(5,2) NOT NULL,
    tenure_months INT NOT NULL,
    start_date DATE DEFAULT (CURRENT_DATE),
    maturity_date DATE,
    maturity_amount DECIMAL(15,2),
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),

    CHECK (principal_amount > 0),
    CHECK (interest_rate >= 0),
    CHECK (tenure_months > 0)
);
```

13 Create RecurringDeposits Table PART I

```sql
CREATE TABLE RecurringDeposits (
    rd_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    account_id INT NOT NULL,
    monthly_amount DECIMAL(12,2) NOT NULL,
    interest_rate DECIMAL(5,2) NOT NULL,
    tenure_months INT NOT NULL,
    start_date DATE DEFAULT (CURRENT_DATE),
    maturity_date DATE,
    maturity_amount DECIMAL(15,2),
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),

    CHECK (monthly_amount > 0),
    CHECK (interest_rate >= 0),
    CHECK (tenure_months > 0)
);
```

14 Create ATMTransactions Table PART I

```sql
CREATE TABLE ATMTransactions (
    atm_transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    card_id INT NOT NULL,
    transaction_type VARCHAR(20) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    atm_location VARCHAR(255),
    transaction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Success',

    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),
    FOREIGN KEY (card_id) REFERENCES Cards(card_id),

    CHECK (amount > 0),
    CHECK (transaction_type IN ('Withdrawal', 'Deposit', 'Balance Inquiry'))
);
```

15 Create KYC Table PART I

```sql
CREATE TABLE KYC (
    kyc_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    document_type VARCHAR(50) NOT NULL,
    document_number VARCHAR(50) UNIQUE NOT NULL,
    verification_date DATE,
    verification_status VARCHAR(20) DEFAULT 'Pending',
    verified_by VARCHAR(100),
    remarks VARCHAR(255),

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),

    CHECK (verification_status IN ('Pending', 'Verified', 'Rejected'))
);
```

16 Create Employees Table PART I

```sql
CREATE TABLE Employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    branch_id INT NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    phone VARCHAR(15) UNIQUE,
    job_title VARCHAR(50),
    salary DECIMAL(12,2),
    hire_date DATE,
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (branch_id) REFERENCES Branches(branch_id),

    CHECK (salary >= 0),
    CHECK (status IN ('Active', 'Inactive'))
);
```

17 Create Complaints Table PART I

```sql
CREATE TABLE Complaints (
    complaint_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    complaint_type VARCHAR(100) NOT NULL,
    complaint_description VARCHAR(500),
    complaint_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    resolution_date DATETIME,
    assigned_employee_id INT,
    priority VARCHAR(20) DEFAULT 'Medium',
    status VARCHAR(20) DEFAULT 'Open',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (assigned_employee_id) REFERENCES Employees(employee_id),

    CHECK (priority IN ('Low', 'Medium', 'High')),
    CHECK (status IN ('Open', 'In Progress', 'Resolved', 'Closed'))
);
```

18 Create InternetBanking Table PART I

```sql
CREATE TABLE InternetBanking (
    internet_banking_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    registration_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login DATETIME,
    status VARCHAR(20) DEFAULT 'Active',

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),

    CHECK (status IN ('Active', 'Blocked', 'Inactive'))
);
```

19 Create AuditLogs Table

PART I

```sql
CREATE TABLE AuditLogs (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    employee_id INT,
    action_type VARCHAR(50) NOT NULL,
    table_name VARCHAR(100),
    record_id INT,
    action_description VARCHAR(500),
    action_date DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)
);
```

PART II

Data Loading & Verification

Seeding every table with realistic sample records, then verifying that all tables exist.

18 INSERT batches	Ordered loads	SHOW TABLES
Seed data	Parents before children	Verification

20 Insert Account Types Data

PART II

```
INSERT INTO AccountTypes
(account_type_name, minimum_balance, interest_rate, description)
VALUES
('Savings', 1000.00, 4.00, 'Regular savings account'),
('Current', 5000.00, 0.00, 'Business current account'),
('Salary', 0.00, 3.50, 'Salary account'),
('Senior Citizen Savings', 1000.00, 5.50, 'Savings account for senior citizens'),
('NRI Savings', 5000.00, 4.50, 'Savings account for NRI customers');
```

5 rows inserted.

21 Insert Branches Data

PART II

```
INSERT INTO Branches
(branch_name, branch_code, address, city, state, pincode, phone, manager_name, opening_date)
VALUES
('Main Branch', 'BR001', 'MG Road', 'Mumbai', 'Maharashtra', '400001', '02240000001', 'Rajesh Sharma',
    '2015-01-10'),
('Andheri Branch', 'BR002', 'Andheri East', 'Mumbai', 'Maharashtra', '400069', '02240000002', 'Priya
    Mehta', '2016-03-15'),
('Bandra Branch', 'BR003', 'Bandra West', 'Mumbai', 'Maharashtra', '400050', '02240000003', 'Amit Verma',
    '2017-06-20'),
('Pune Branch', 'BR004', 'FC Road', 'Pune', 'Maharashtra', '411004', '02040000004', 'Neha Kapoor',
    '2018-02-12'),
('Delhi Branch', 'BR005', 'Connaught Place', 'New Delhi', 'Delhi', '110001', '01140000005', 'Vikas Singh',
    '2019-09-05'),
('Bangalore Branch', 'BR006', 'MG Road', 'Bangalore', 'Karnataka', '560001', '08040000006', 'Anjali Rao',
    '2020-01-18');
```

6 rows inserted.

22 Insert Employees Data PART II

```sql
INSERT INTO Employees
(branch_id, first_name, last_name, email, phone, job_title, salary, hire_date)
VALUES
(1, 'Rahul', 'Sharma', 'rahul@bank.com', '9000000001', 'Branch Manager', 85000, '2018-04-10'),
(1, 'Sneha', 'Patel', 'sneha@bank.com', '9000000002', 'Cashier', 45000, '2020-07-15'),
(2, 'Aman', 'Khan', 'aman@bank.com', '9000000003', 'Branch Manager', 82000, '2019-02-20'),
(2, 'Pooja', 'Gupta', 'pooja@bank.com', '9000000004', 'Loan Officer', 60000, '2021-05-12'),
(3, 'Rohit', 'Verma', 'rohit@bank.com', '9000000005', 'Branch Manager', 80000, '2019-08-18'),
(3, 'Kavita', 'Joshi', 'kavita@bank.com', '9000000006', 'Customer Service', 48000, '2022-01-10'),
(4, 'Arjun', 'Mehta', 'arjun@bank.com', '9000000007', 'Branch Manager', 78000, '2020-03-25'),
(5, 'Nisha', 'Singh', 'nisha@bank.com', '9000000008', 'Loan Officer', 62000, '2021-11-05'),
(6, 'Karan', 'Rao', 'karan@bank.com', '9000000009', 'Cashier', 46000, '2022-06-14');
```

9 rows inserted.

23 Insert Customers Data PART II

```sql
INSERT INTO Customers
(first_name, last_name, email, phone, date_of_birth, gender, address, city, state, pincode)
VALUES
('Aarav', 'Sharma', 'aarav@gmail.com', '9876500001', '1995-05-12', 'Male', 'MG Road', 'Mumbai',
    'Maharashtra', '400001'),
('Ananya', 'Patel', 'ananya@gmail.com', '9876500002', '1998-08-22', 'Female', 'Andheri East', 'Mumbai',
    'Maharashtra', '400069'),
('Rohan', 'Verma', 'rohan@gmail.com', '9876500003', '1992-02-15', 'Male', 'Bandra West', 'Mumbai',
    'Maharashtra', '400050'),
('Simran', 'Khan', 'simran@gmail.com', '9876500004', '1997-11-03', 'Female', 'FC Road', 'Pune',
    'Maharashtra', '411004'),
('Aditya', 'Mehta', 'aditya@gmail.com', '9876500005', '1990-06-18', 'Male', 'Connaught Place', 'New
    Delhi', 'Delhi', '110001'),
('Isha', 'Gupta', 'isha@gmail.com', '9876500006', '1999-01-27', 'Female', 'MG Road', 'Bangalore',
    'Karnataka', '560001'),
('Kabir', 'Joshi', 'kabir@gmail.com', '9876500007', '1994-09-10', 'Male', 'Powai', 'Mumbai',
    'Maharashtra', '400076'),
('Meera', 'Singh', 'meera@gmail.com', '9876500008', '1996-12-05', 'Female', 'Viman Nagar', 'Pune',
    'Maharashtra', '411014'),
('Vivek', 'Rao', 'vivek@gmail.com', '9876500009', '1988-04-30', 'Male', 'Indiranagar', 'Bangalore',
    'Karnataka', '560038'),
('Diya', 'Kapoor', 'diya@gmail.com', '9876500010', '2000-07-19', 'Female', 'Gurgaon Road', 'New Delhi',
    'Delhi', '110037');
```

10 rows inserted.

24 Insert Accounts Data PART II

```sql
INSERT INTO Accounts
(customer_id, branch_id, account_type_id, account_number, balance, opening_date)
VALUES
(1, 1, 1, 'ACC100001', 75000.00, '2022-01-10'),
(2, 2, 1, 'ACC100002', 45000.00, '2022-03-15'),
(3, 3, 2, 'ACC100003', 150000.00, '2021-07-20'),
(4, 4, 1, 'ACC100004', 60000.00, '2023-01-05'),
(5, 5, 3, 'ACC100005', 85000.00, '2022-08-18'),
(6, 6, 1, 'ACC100006', 55000.00, '2023-04-12'),
(7, 1, 4, 'ACC100007', 120000.00, '2021-11-25'),
(8, 4, 1, 'ACC100008', 35000.00, '2023-06-10'),
(9, 6, 2, 'ACC100009', 200000.00, '2020-09-15'),
(10, 5, 5, 'ACC100010', 95000.00, '2024-01-20');
```

10 rows inserted.

25 Insert Transactions Data PART II

```sql
INSERT INTO Transactions
(account_id, transaction_type, amount, description, reference_number)
VALUES
(1, 'Deposit', 25000.00, 'Cash Deposit', 'TXN100001'),
(1, 'Withdrawal', 5000.00, 'ATM Withdrawal', 'TXN100002'),
(2, 'Deposit', 15000.00, 'Salary Credit', 'TXN100003'),
(2, 'Withdrawal', 3000.00, 'Cash Withdrawal', 'TXN100004'),
(3, 'Deposit', 50000.00, 'Business Deposit', 'TXN100005'),
(3, 'Transfer', 10000.00, 'Fund Transfer', 'TXN100006'),
(4, 'Deposit', 20000.00, 'Cash Deposit', 'TXN100007'),
(5, 'Deposit', 30000.00, 'Salary Credit', 'TXN100008'),
(6, 'Withdrawal', 7000.00, 'ATM Withdrawal', 'TXN100009'),
(7, 'Deposit', 40000.00, 'Cash Deposit', 'TXN100010'),
(8, 'Withdrawal', 5000.00, 'Online Payment', 'TXN100011'),
(9, 'Deposit', 75000.00, 'Business Deposit', 'TXN100012'),
(10, 'Transfer', 15000.00, 'Fund Transfer', 'TXN100013');
```

13 rows inserted.

26 Insert Beneficiaries Data PART II

```sql
INSERT INTO Beneficiaries
(customer_id, beneficiary_name, account_number, bank_name, ifsc_code, nickname)
VALUES
(1, 'Ananya Patel', 'ACC100002', 'BankingSystem Bank', 'BSBK000001', 'Ananya'),
(2, 'Rohan Verma', 'ACC100003', 'BankingSystem Bank', 'BSBK000002', 'Rohan'),
(3, 'Simran Khan', 'ACC100004', 'BankingSystem Bank', 'BSBK000003', 'Simran'),
(4, 'Aditya Mehta', 'ACC100005', 'BankingSystem Bank', 'BSBK000004', 'Aditya'),
(5, 'Isha Gupta', 'ACC100006', 'BankingSystem Bank', 'BSBK000005', 'Isha'),
(6, 'Kabir Joshi', 'ACC100007', 'BankingSystem Bank', 'BSBK000006', 'Kabir'),
(7, 'Meera Singh', 'ACC100008', 'BankingSystem Bank', 'BSBK000007', 'Meera'),
(8, 'Vivek Rao', 'ACC100009', 'BankingSystem Bank', 'BSBK000008', 'Vivek'),
(9, 'Diya Kapoor', 'ACC100010', 'BankingSystem Bank', 'BSBK000009', 'Diya'),
(10, 'Aarav Sharma', 'ACC100001', 'BankingSystem Bank', 'BSBK000010', 'Aarav');
```

10 rows inserted.

27 Create FundTransfers Table PART II

Re-declares the FundTransfers table already created in Step 8. A MySQL run reports "Table already exists" and continues; the table is unchanged.

```sql
CREATE TABLE FundTransfers (
    transfer_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_account_id INT NOT NULL,
    receiver_account_id INT NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    transfer_type VARCHAR(20) NOT NULL,
    transfer_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    reference_number VARCHAR(50) UNIQUE,
    remarks VARCHAR(255),
    status VARCHAR(20) DEFAULT 'Success',

    FOREIGN KEY (sender_account_id) REFERENCES Accounts(account_id),
    FOREIGN KEY (receiver_account_id) REFERENCES Accounts(account_id),

    CHECK (amount > 0),
    CHECK (transfer_type IN ('NEFT', 'RTGS', 'IMPS', 'UPI'))
);
```

28 Insert Loans Data PART II

```sql
INSERT INTO Loans
(customer_id, branch_id, loan_type, loan_amount, interest_rate, tenure_months, start_date, end_date)
VALUES
(1, 1, 'Home Loan', 2500000.00, 8.50, 240, '2024-01-10', '2044-01-10'),
(2, 2, 'Personal Loan', 500000.00, 11.00, 60, '2024-02-15', '2029-02-15'),
(3, 3, 'Business Loan', 5000000.00, 9.25, 120, '2023-06-20', '2033-06-20'),
(4, 4, 'Education Loan', 800000.00, 7.50, 84, '2024-03-05', '2031-03-05'),
(5, 5, 'Car Loan', 1200000.00, 9.00, 60, '2023-08-18', '2028-08-18'),
(6, 6, 'Personal Loan', 300000.00, 10.50, 36, '2024-04-12', '2027-04-12'),
(7, 1, 'Home Loan', 3500000.00, 8.25, 180, '2022-11-25', '2037-11-25'),
(8, 4, 'Education Loan', 600000.00, 7.25, 60, '2024-06-10', '2029-06-10'),
(9, 6, 'Business Loan', 4000000.00, 9.50, 120, '2022-09-15', '2032-09-15'),
(10, 5, 'Car Loan', 900000.00, 8.75, 60, '2024-01-20', '2029-01-20');
```

10 rows inserted.

29 Insert Loan Payments Data PART II

```sql
INSERT INTO LoanPayments
(loan_id, payment_amount, payment_method, transaction_reference, remarks)
VALUES
(1, 25000.00, 'Bank Transfer', 'LP100001', 'Monthly EMI'),
(1, 25000.00, 'Bank Transfer', 'LP100002', 'Monthly EMI'),
(2, 11000.00, 'UPI', 'LP100003', 'Monthly EMI'),
(3, 65000.00, 'Bank Transfer', 'LP100004', 'Monthly EMI'),
(4, 12000.00, 'UPI', 'LP100005', 'Monthly EMI'),
(5, 25000.00, 'Bank Transfer', 'LP100006', 'Monthly EMI'),
(6, 10000.00, 'UPI', 'LP100007', 'Monthly EMI'),
(7, 35000.00, 'Bank Transfer', 'LP100008', 'Monthly EMI'),
(8, 12000.00, 'UPI', 'LP100009', 'Monthly EMI'),
(9, 55000.00, 'Bank Transfer', 'LP100010', 'Monthly EMI'),
(10, 19000.00, 'UPI', 'LP100011', 'Monthly EMI');
```

11 rows inserted.

30 Insert Cards Data

PART II

```sql
INSERT INTO Cards
(customer_id, account_id, card_type, card_number, issue_date, expiry_date, daily_limit)
VALUES
(1, 1, 'Debit', '4500123456780001', '2022-01-15', '2027-01-15', 50000),
(2, 2, 'Debit', '4500123456780002', '2022-03-20', '2027-03-20', 50000),
(3, 3, 'Credit', '4500123456780003', '2021-07-25', '2026-07-25', 100000),
(4, 4, 'Debit', '4500123456780004', '2023-01-10', '2028-01-10', 50000),
(5, 5, 'Credit', '4500123456780005', '2022-08-25', '2027-08-25', 150000),
(6, 6, 'Debit', '4500123456780006', '2023-04-15', '2028-04-15', 50000),
(7, 7, 'Debit', '4500123456780007', '2021-12-01', '2026-12-01', 75000),
(8, 8, 'Debit', '4500123456780008', '2023-06-15', '2028-06-15', 50000),
(9, 9, 'Credit', '4500123456780009', '2020-09-20', '2025-09-20', 150000),
(10, 10, 'Debit', '4500123456780010', '2024-01-25', '2029-01-25', 50000);
```

10 rows inserted.

31 Insert Fixed Deposits Data

PART II

```sql
INSERT INTO FixedDeposits
(customer_id, account_id, principal_amount, interest_rate, tenure_months, start_date, maturity_date,
    maturity_amount)
VALUES
(1, 1, 100000.00, 6.50, 12, '2024-01-10', '2025-01-10', 106500.00),
(2, 2, 150000.00, 6.75, 24, '2024-02-15', '2026-02-15', 170250.00),
(3, 3, 500000.00, 7.00, 36, '2023-06-20', '2026-06-20', 605000.00),
(4, 4, 200000.00, 6.50, 12, '2024-03-05', '2025-03-05', 213000.00),
(5, 5, 300000.00, 7.25, 24, '2023-08-18', '2025-08-18', 343500.00),
(6, 6, 100000.00, 6.50, 12, '2024-04-12', '2025-04-12', 106500.00),
(7, 7, 400000.00, 7.50, 36, '2022-11-25', '2025-11-25', 490000.00),
(8, 8, 175000.00, 6.75, 24, '2024-06-10', '2026-06-10', 198625.00),
(9, 9, 600000.00, 7.25, 36, '2022-09-15', '2025-09-15', 730500.00),
(10, 10, 250000.00, 7.00, 12, '2024-01-20', '2025-01-20', 267500.00);
```

10 rows inserted.

32 Insert Recurring Deposits Data

PART II

```sql
INSERT INTO RecurringDeposits
(customer_id, account_id, monthly_amount, interest_rate, tenure_months, start_date, maturity_date,
    maturity_amount)
VALUES
(1, 1, 5000.00, 6.50, 12, '2024-01-10', '2025-01-10', 62000.00),
(2, 2, 7500.00, 6.75, 24, '2024-02-15', '2026-02-15', 190000.00),
(3, 3, 10000.00, 7.00, 36, '2023-06-20', '2026-06-20', 390000.00),
(4, 4, 4000.00, 6.50, 12, '2024-03-05', '2025-03-05', 49600.00),
(5, 5, 8000.00, 7.25, 24, '2023-08-18', '2025-08-18', 210000.00),
(6, 6, 5000.00, 6.50, 12, '2024-04-12', '2025-04-12', 62000.00),
(7, 7, 12000.00, 7.50, 36, '2022-11-25', '2025-11-25', 480000.00),
(8, 8, 6000.00, 6.75, 24, '2024-06-10', '2026-06-10', 152000.00),
(9, 9, 15000.00, 7.25, 36, '2022-09-15', '2025-09-15', 585000.00),
(10, 10, 7000.00, 7.00, 12, '2024-01-20', '2025-01-20', 86800.00);
```

10 rows inserted.

33 Create ATMTransactions Table PART II

Re-declares the ATMTransactions table already created in Step 14. A MySQL run reports "Table already exists" and continues; the table is unchanged.

```sql
CREATE TABLE ATMTransactions (
    atm_transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL,
    card_id INT NOT NULL,
    transaction_type VARCHAR(20) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    atm_location VARCHAR(255),
    transaction_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Success',

    FOREIGN KEY (account_id) REFERENCES Accounts(account_id),
    FOREIGN KEY (card_id) REFERENCES Cards(card_id),

    CHECK (amount > 0),
    CHECK (transaction_type IN ('Withdrawal', 'Deposit', 'Balance Inquiry'))
);
```

34 Insert KYC Data PART II

```sql
INSERT INTO KYC
(customer_id, document_type, document_number, verification_date, verification_status, verified_by)
VALUES
(1, 'Aadhaar', 'AAD100001', '2024-01-15', 'Verified', 'Rahul Sharma'),
(2, 'PAN', 'PAN100002', '2024-02-20', 'Verified', 'Aman Khan'),
(3, 'Aadhaar', 'AAD100003', '2024-03-10', 'Verified', 'Rohit Verma'),
(4, 'Passport', 'PASS100004', '2024-03-15', 'Verified', 'Arjun Mehta'),
(5, 'PAN', 'PAN100005', '2024-04-05', 'Verified', 'Nisha Singh'),
(6, 'Aadhaar', 'AAD100006', '2024-04-20', 'Verified', 'Karan Rao'),
(7, 'PAN', 'PAN100007', '2024-05-10', 'Pending', NULL),
(8, 'Aadhaar', 'AAD100008', '2024-05-15', 'Verified', 'Arjun Mehta'),
(9, 'Passport', 'PASS100009', '2024-06-01', 'Verified', 'Karan Rao'),
(10, 'PAN', 'PAN100010', '2024-06-10', 'Pending', NULL);
```

10 rows inserted.

35 Insert Complaints Data PART II

```sql
INSERT INTO Complaints
(customer_id, complaint_type, complaint_description, assigned_employee_id, priority, status)
VALUES
(1, 'ATM Issue', 'ATM cash withdrawal issue', 2, 'High', 'Resolved'),
(2, 'Transaction Issue', 'Transaction was not reflected', 3, 'Medium', 'In Progress'),
(3, 'Card Issue', 'Credit card payment issue', 5, 'High', 'Open'),
(4, 'Account Issue', 'Unable to update account details', 7, 'Low', 'Resolved'),
(5, 'Loan Issue', 'Loan EMI clarification required', 8, 'Medium', 'In Progress'),
(6, 'Internet Banking', 'Unable to login to internet banking', 9, 'High', 'Open'),
(7, 'Fund Transfer', 'Fund transfer pending', 1, 'High', 'Resolved'),
(8, 'ATM Issue', 'ATM cash deposit issue', 7, 'Medium', 'Open'),
(9, 'Loan Issue', 'Loan payment not updated', 9, 'Medium', 'In Progress'),
(10, 'Card Issue', 'Debit card transaction declined', 8, 'High', 'Resolved');
```

10 rows inserted.

36 Insert Internet Banking Data PART II

```sql
INSERT INTO InternetBanking
(customer_id, username, password_hash, last_login)
VALUES
(1, 'aarav123', 'HASH001', '2026-09-10 10:30:00'),
(2, 'ananya123', 'HASH002', '2026-09-11 11:15:00'),
(3, 'rohan123', 'HASH003', '2026-09-12 09:45:00'),
(4, 'simran123', 'HASH004', '2026-09-12 14:20:00'),
(5, 'aditya123', 'HASH005', '2026-09-13 16:10:00'),
(6, 'isha123', 'HASH006', '2026-09-14 12:05:00'),
(7, 'kabir123', 'HASH007', '2026-09-14 18:30:00'),
(8, 'meera123', 'HASH008', '2026-09-15 10:15:00'),
(9, 'vivek123', 'HASH009', '2026-09-16 13:40:00'),
(10, 'diya123', 'HASH010', '2026-09-17 17:25:00');
```

10 rows inserted.

37 Insert Audit Logs Data PART II

```sql
INSERT INTO AuditLogs
(customer_id, employee_id, action_type, table_name, record_id, action_description)
VALUES
(1, 1, 'INSERT', 'Accounts', 1, 'New bank account created'),
(2, 3, 'UPDATE', 'Customers', 2, 'Customer information updated'),
(3, 5, 'INSERT', 'Loans', 3, 'New business loan created'),
(4, 7, 'UPDATE', 'Accounts', 4, 'Account status updated'),
(5, 8, 'INSERT', 'LoanPayments', 5, 'Loan payment recorded'),
(6, 9, 'LOGIN', 'InternetBanking', 6, 'Customer logged into internet banking'),
(7, 1, 'TRANSFER', 'FundTransfers', 7, 'Fund transfer completed'),
(8, 7, 'INSERT', 'Complaints', 8, 'New customer complaint created'),
(9, 9, 'PAYMENT', 'LoanPayments', 9, 'Loan payment processed'),
(10, 8, 'UPDATE', 'KYC', 10, 'KYC information updated');
```

10 rows inserted.

38 Verify All Tables PART II

SQL **Verify All Tables**

```sql
SHOW TABLES;
```

OUTPUT · 18 ROWS

Tables_in_bankingsystem
Accounts
AccountTypes
ATMTransactions
AuditLogs
Beneficiaries
Branches
Cards
Complaints
Customers
Employees

Tables_in_bankingsystem
FixedDeposits
FundTransfers
InternetBanking
KYC
LoanPayments
Loans
RecurringDeposits
Transactions

PART III

Querying & Analysis

From basic SELECTs to joins, subqueries, common table expressions, set operations and window functions.

Filtering
Operators and patterns

Joins
Inner / left / right / self

Window functions
Ranking and running totals

39 Basic SELECT Queries

PART III

Q1 Display all customers.

```
SELECT * FROM Customers;
```

OUTPUT · 10 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
1	Aarav	Sharma	aarav@gmail.com	9876500001	1995-05-12	Male	MG Road	Mumbai	Maharashtra	400001	2026-10-29	Active
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
3	Rohan	Verma	rohan@gmail.com	9876500003	1992-02-15	Male	Bandra West	Mumbai	Maharashtra	400050	2026-10-29	Active
4	Simran	Khan	simran@gmail.com	9876500004	1997-11-03	Female	FC Road	Pune	Maharashtra	411004	2026-10-29	Active
5	Aditya	Mehta	aditya@gmail.com	9876500005	1990-06-18	Male	Connaught Place	New Delhi	Delhi	110001	2026-10-29	Active
6	Isha	Gupta	isha@gmail.com	9876500006	1999-01-27	Female	MG Road	Bangalore	Karnataka	560001	2026-10-29	Active
7	Kabir	Joshi	kabir@gmail.com	9876500007	1994-09-10	Male	Powai	Mumbai	Maharashtra	400076	2026-10-29	Active
8	Meera	Singh	meera@gmail.com	9876500008	1996-12-05	Female	Viman Nagar	Pune	Maharashtra	411014	2026-10-29	Active
9	Vivek	Rao	vivek@gmail.com	9876500009	1988-04-30	Male	Indiranagar	Bangalore	Karnataka	560038	2026-10-29	Active
10	Diya	Kapoor	diya@gmail.com	9876500010	2000-07-19	Female	Gurgaon Road	New Delhi	Delhi	110037	2026-10-29	Active

Q2 Display the first name, last name, and email of all customers.

```
SELECT first_name, last_name, email
FROM Customers;
```

OUTPUT · 10 ROWS

first_name	last_name	email
Aarav	Sharma	aarav@gmail.com
Ananya	Patel	ananya@gmail.com
Rohan	Verma	rohan@gmail.com
Simran	Khan	simran@gmail.com
Aditya	Mehta	aditya@gmail.com
Isha	Gupta	isha@gmail.com
Kabir	Joshi	kabir@gmail.com
Meera	Singh	meera@gmail.com
Vivek	Rao	vivek@gmail.com
Diya	Kapoor	diya@gmail.com

Q3 Find all customers who live in Mumbai.

```
SELECT *
FROM Customers
WHERE city = 'Mumbai';
```

OUTPUT · 4 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
1	Aarav	Sharma	aarav@gmail.com	9876500001	1995-05-12	Male	MG Road	Mumbai	Maharashtra	400001	2026-10-29	Active
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
3	Rohan	Verma	rohan@gmail.com	9876500003	1992-02-15	Male	Bandra West	Mumbai	Maharashtra	400050	2026-10-29	Active
7	Kabir	Joshi	kabir@gmail.com	9876500007	1994-09-10	Male	Powai	Mumbai	Maharashtra	400076	2026-10-29	Active

Q4 Find all active customers.

```
SELECT *
FROM Customers
WHERE status = 'Active';
```

OUTPUT · 10 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
1	Aarav	Sharma	aarav@gmail.com	9876500001	1995-05-12	Male	MG Road	Mumbai	Maharashtra	400001	2026-10-29	Active
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
3	Rohan	Verma	rohan@gmail.com	9876500003	1992-02-15	Male	Bandra West	Mumbai	Maharashtra	400050	2026-10-29	Active
4	Simran	Khan	simran@gmail.com	9876500004	1997-11-03	Female	FC Road	Pune	Maharashtra	411004	2026-10-29	Active
5	Aditya	Mehta	aditya@gmail.com	9876500005	1990-06-18	Male	Connaught Place	New Delhi	Delhi	110001	2026-10-29	Active
6	Isha	Gupta	isha@gmail.com	9876500006	1999-01-27	Female	MG Road	Bangalore	Karnataka	560001	2026-10-29	Active
7	Kabir	Joshi	kabir@gmail.com	9876500007	1994-09-10	Male	Powai	Mumbai	Maharashtra	400076	2026-10-29	Active
8	Meera	Singh	meera@gmail.com	9876500008	1996-12-05	Female	Viman Nagar	Pune	Maharashtra	411014	2026-10-29	Active
9	Vivek	Rao	vivek@gmail.com	9876500009	1988-04-30	Male	Indiranagar	Bangalore	Karnataka	560038	2026-10-29	Active
10	Diya	Kapoor	diya@gmail.com	9876500010	2000-07-19	Female	Gurgaon Road	New Delhi	Delhi	110037	2026-10-29	Active

Q5 Display all accounts in descending order of balance.

```
SELECT *
FROM Accounts
ORDER BY balance DESC;
```

OUTPUT · 10 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
9	9	6	2	ACC100009	200000.00	2020-09-15	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
2	2	2	1	ACC100002	45000.00	2022-03-15	Active

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
8	8	4	1	ACC100008	35000.00	2023-06-10	Active

Q6 Find the top 5 accounts with the highest balance.

```
SELECT *
FROM Accounts
ORDER BY balance DESC
LIMIT 5;
```

OUTPUT · 5 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
9	9	6	2	ACC100009	200000.00	2020-09-15	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active

40 Filtering with Operators PART III

Q1 Find accounts with a balance greater than 100,000.

```
SELECT *
FROM Accounts
WHERE balance > 100000;
```

OUTPUT · 3 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
9	9	6	2	ACC100009	200000.00	2020-09-15	Active

Q2 Find accounts with a balance greater than or equal to 50,000.

```
SELECT *
FROM Accounts
WHERE balance >= 50000;
```

OUTPUT · 8 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
9	9	6	2	ACC100009	200000.00	2020-09-15	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active

Q3 Find accounts with a balance less than 50,000.

```
SELECT *
FROM Accounts
WHERE balance < 50000;
```

OUTPUT · 2 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
2	2	2	1	ACC100002	45000.00	2022-03-15	Active
8	8	4	1	ACC100008	35000.00	2023-06-10	Active

Q4 Find accounts whose balance is not equal to 50,000.

```
SELECT *
FROM Accounts
WHERE balance <> 50000;
```

OUTPUT · 10 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
2	2	2	1	ACC100002	45000.00	2022-03-15	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
8	8	4	1	ACC100008	35000.00	2023-06-10	Active
9	9	6	2	ACC100009	200000.00	2020-09-15	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active

Q5 Find accounts with a balance between 50,000 and 150,000.

```
SELECT *
FROM Accounts
WHERE balance BETWEEN 50000 AND 150000;
```

OUTPUT · 7 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active

Q6 Find customers who live in Mumbai or Pune.

```
SELECT *
FROM Customers
WHERE city IN ('Mumbai', 'Pune');
```

OUTPUT · 6 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
1	Aarav	Sharma	aarav@gmail.com	9876500001	1995-05-12	Male	MG Road	Mumbai	Maharashtra	400001	2026-10-29	Active
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
3	Rohan	Verma	rohan@gmail.com	9876500003	1992-02-15	Male	Bandra West	Mumbai	Maharashtra	400050	2026-10-29	Active
4	Simran	Khan	simran@gmail.com	9876500004	1997-11-03	Female	FC Road	Pune	Maharashtra	411004	2026-10-29	Active
7	Kabir	Joshi	kabir@gmail.com	9876500007	1994-09-10	Male	Powai	Mumbai	Maharashtra	400076	2026-10-29	Active
8	Meera	Singh	meera@gmail.com	9876500008	1996-12-05	Female	Viman Nagar	Pune	Maharashtra	411014	2026-10-29	Active

Q7 Find customers who do not live in Mumbai or Pune.

```
SELECT *
FROM Customers
WHERE city NOT IN ('Mumbai', 'Pune');
```

OUTPUT · 4 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
5	Aditya	Mehta	aditya@gmail.com	9876500005	1990-06-18	Male	Connaught Place	New Delhi	Delhi	110001	2026-10-29	Active
6	Isha	Gupta	isha@gmail.com	9876500006	1999-01-27	Female	MG Road	Bangalore	Karnataka	560001	2026-10-29	Active
9	Vivek	Rao	vivek@gmail.com	9876500009	1988-04-30	Male	Indiranagar	Bangalore	Karnataka	560038	2026-10-29	Active
10	Diya	Kapoor	diya@gmail.com	9876500010	2000-07-19	Female	Gurgaon Road	New Delhi	Delhi	110037	2026-10-29	Active

Q8 Find customers whose first name starts with 'A'.

```
SELECT *
FROM Customers
WHERE first_name LIKE 'A%';
```

OUTPUT · 3 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
1	Aarav	Sharma	aarav@gmail.com	9876500001	1995-05-12	Male	MG Road	Mumbai	Maharashtra	400001	2026-10-29	Active
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
5	Aditya	Mehta	aditya@gmail.com	9876500005	1990-06-18	Male	Connaught Place	New Delhi	Delhi	110001	2026-10-29	Active

Q9 Find customers whose first name ends with 'a'.

```
SELECT *
FROM Customers
WHERE first_name LIKE '%a';
```

OUTPUT · 5 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
5	Aditya	Mehta	aditya@gmail.com	9876500005	1990-06-18	Male	Connaught Place	New Delhi	Delhi	110001	2026-10-29	Active
6	Isha	Gupta	isha@gmail.com	9876500006	1999-01-27	Female	MG Road	Bangalore	Karnataka	560001	2026-10-29	Active
8	Meera	Singh	meera@gmail.com	9876500008	1996-12-05	Female	Viman Nagar	Pune	Maharashtra	411014	2026-10-29	Active
10	Diya	Kapoor	diya@gmail.com	9876500010	2000-07-19	Female	Gurgaon Road	New Delhi	Delhi	110037	2026-10-29	Active

Q10 Find customers whose first name contains 'an'.

```
SELECT *
FROM Customers
WHERE first_name LIKE '%an%';
```

OUTPUT · 3 ROWS

customer_id	first_name	last_name	email	phone	date_of_birth	gender	address	city	state	pincode	registration_date	status
2	Ananya	Patel	ananya@gmail.com	9876500002	1998-08-22	Female	Andheri East	Mumbai	Maharashtra	400069	2026-10-29	Active
3	Rohan	Verma	rohan@gmail.com	9876500003	1992-02-15	Male	Bandra West	Mumbai	Maharashtra	400050	2026-10-29	Active
4	Simran	Khan	simran@gmail.com	9876500004	1997-11-03	Female	FC Road	Pune	Maharashtra	411004	2026-10-29	Active

Q11 Find active accounts with a balance greater than 50,000.

```
SELECT *
FROM Accounts
WHERE balance > 50000
AND status = 'Active';
```

OUTPUT · 8 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
5	5	5	3	ACC100005	85000.00	2022-08-18	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
9	9	6	2	ACC100009	200000.00	2020-09-15	Active
10	10	5	5	ACC100010	95000.00	2024-01-20	Active

Q12 Find accounts with a balance greater than 100,000 or accounts of type 1.

```
SELECT *
FROM Accounts
WHERE balance > 100000
OR account_type_id = 1;
```

OUTPUT · 8 ROWS

account_id	customer_id	branch_id	account_type_id	account_number	balance	opening_date	status
1	1	1	1	ACC100001	75000.00	2022-01-10	Active
2	2	2	1	ACC100002	45000.00	2022-03-15	Active
3	3	3	2	ACC100003	150000.00	2021-07-20	Active
4	4	4	1	ACC100004	60000.00	2023-01-05	Active
6	6	6	1	ACC100006	55000.00	2023-04-12	Active
7	7	1	4	ACC100007	120000.00	2021-11-25	Active
8	8	4	1	ACC100008	35000.00	2023-06-10	Active
9	9	6	2	ACC100009	200000.00	2020-09-15	Active

41 SQL Functions

PART III

COUNT, SUM, AVG, MIN, MAX, UPPER, LOWER, CONCAT, LENGTH, ROUND.

Q1 Find the total number of customers.

```
SELECT COUNT(*) AS total_customers
FROM Customers;
```

OUTPUT · 1 ROW

total_customers
10

Q2 Find the total balance of all accounts.

```
SELECT SUM(balance) AS total_balance
FROM Accounts;
```

OUTPUT · 1 ROW

total_balance
920000.00

Q3 Find the average account balance.

```
SELECT AVG(balance) AS average_balance
FROM Accounts;
```

OUTPUT · 1 ROW

average_balance
92000.000000

Q4 Find the minimum account balance.

```
SELECT MIN(balance) AS minimum_balance
FROM Accounts;
```

OUTPUT · 1 ROW

minimum_balance
35000.00

Q5 Find the maximum account balance.

```
SELECT MAX(balance) AS maximum_balance
FROM Accounts;
```

OUTPUT · 1 ROW

maximum_balance
200000.00

Q6 Display first names in uppercase and last names in lowercase.

```
SELECT
    UPPER(first_name) AS first_name_upper,
    LOWER(last_name) AS last_name_lower
FROM Customers;
```

OUTPUT · 10 ROWS

first_name_upper	last_name_lower
AARAV	sharma
ANANYA	patel
ROHAN	verma
SIMRAN	khan
ADITYA	mehta
ISHA	gupta
KABIR	joshi
MEERA	singh
VIVEK	rao
DIYA	kapoor

Q7 **Display the full name of each customer.**

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name
FROM Customers;
```

OUTPUT · 10 ROWS

full_name
Aarav Sharma
Ananya Patel
Rohan Verma
Simran Khan
Aditya Mehta
Isha Gupta
Kabir Joshi
Meera Singh
Vivek Rao
Diya Kapoor

Q8 **Find the length of each customer's first name.**

```
SELECT
    first_name,
    LENGTH(first_name) AS name_length
FROM Customers;
```

OUTPUT · 10 ROWS

first_name	name_length
Aarav	5
Ananya	6
Rohan	5
Simran	6
Aditya	6
Isha	4
Kabir	5
Meera	5
Vivek	5
Diya	4

Q9 Display account balances in thousands, rounded to 2 decimal places.

```sql
SELECT
    account_id,
    balance,
    ROUND(balance / 1000, 2) AS balance_in_thousands
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	balance	balance_in_thousands
1	75000.00	75.00
2	45000.00	45.00
3	150000.00	150.00
4	60000.00	60.00
5	85000.00	85.00
6	55000.00	55.00
7	120000.00	120.00
8	35000.00	35.00
9	200000.00	200.00
10	95000.00	95.00

42 GROUP BY and HAVING

PART III

Q1 Count the number of customers in each city.

```sql
SELECT city, COUNT(*) AS total_customers
FROM Customers
GROUP BY city;
```

OUTPUT · 4 ROWS

city	total_customers
Bangalore	2
Mumbai	4
New Delhi	2
Pune	2

Q2 **Count the number of accounts in each branch.**

```
SELECT branch_id, COUNT(*) AS total_accounts
FROM Accounts
GROUP BY branch_id;
```

OUTPUT · 6 ROWS

branch_id	total_accounts
1	2
2	1
3	1
4	2
5	2
6	2

Q3 **Find the average balance for each account type.**

```
SELECT account_type_id, AVG(balance) AS average_balance
FROM Accounts
GROUP BY account_type_id;
```

OUTPUT · 5 ROWS

account_type_id	average_balance
1	54000.000000
2	175000.000000
3	85000.000000
4	120000.000000
5	95000.000000

Q4 **Find the total transaction amount for each transaction type.**

```
SELECT transaction_type, SUM(amount) AS total_amount
FROM Transactions
GROUP BY transaction_type;
```

OUTPUT · 3 ROWS

transaction_type	total_amount
Deposit	255000.00
Transfer	25000.00
Withdrawal	20000.00

Q5 Find transaction types having more than 2 transactions.

```sql
SELECT transaction_type, COUNT(*) AS total_transactions
FROM Transactions
GROUP BY transaction_type
HAVING COUNT(*) > 2;
```

OUTPUT · 2 ROWS

transaction_type	total_transactions
Deposit	7
Withdrawal	4

Q6 Find branches whose total account balance is greater than 100,000.

```sql
SELECT branch_id, SUM(balance) AS total_balance
FROM Accounts
GROUP BY branch_id
HAVING SUM(balance) > 100000;
```

OUTPUT · 4 ROWS

branch_id	total_balance
1	195000.00
3	150000.00
5	180000.00
6	255000.00

43 DISTINCT and NULL Handling

PART III

Q1 Find all unique cities of customers.

```sql
SELECT DISTINCT city
FROM Customers;
```

OUTPUT · 4 ROWS

city
Mumbai
Pune
New Delhi
Bangalore

Q2 Find all unique states of customers.

```
SELECT DISTINCT state
FROM Customers;
```

OUTPUT · 3 ROWS

state
Maharashtra
Delhi
Karnataka

Q3 Find all unique transaction types.

```
SELECT DISTINCT transaction_type
FROM Transactions;
```

OUTPUT · 3 ROWS

transaction_type
Deposit
Withdrawal
Transfer

Q4 Find KYC records where no employee has verified the document.

```
SELECT *
FROM KYC
WHERE verified_by IS NULL;
```

OUTPUT · 2 ROWS

kyc_id	customer_id	document_type	document_number	verification_date	verification_status	verified_by	remarks
7	7	PAN	PAN100007	2024-05-10	Pending	*NULL*	*NULL*
10	10	PAN	PAN100010	2024-06-10	Pending	*NULL*	*NULL*

Q5 Find KYC records where an employee has verified the document.

```
SELECT *
FROM KYC
WHERE verified_by IS NOT NULL;
```

OUTPUT · 8 ROWS

kyc_id	customer_id	document_type	document_number	verification_date	verification_status	verified_by	remarks
1	1	Aadhaar	AAD100001	2024-01-15	Verified	Rahul Sharma	*NULL*
2	2	PAN	PAN100002	2024-02-20	Verified	Aman Khan	*NULL*
3	3	Aadhaar	AAD100003	2024-03-10	Verified	Rohit Verma	*NULL*
4	4	Passport	PASS100004	2024-03-15	Verified	Arjun Mehta	*NULL*
5	5	PAN	PAN100005	2024-04-05	Verified	Nisha Singh	*NULL*
6	6	Aadhaar	AAD100006	2024-04-20	Verified	Karan Rao	*NULL*
8	8	Aadhaar	AAD100008	2024-05-15	Verified	Arjun Mehta	*NULL*
9	9	Passport	PASS100009	2024-06-01	Verified	Karan Rao	*NULL*

Q6 Display 'Not Assigned' when the KYC verifier is NULL.

```
SELECT
    customer_id,
    document_type,
    COALESCE(verified_by, 'Not Assigned') AS verified_by
FROM KYC;
```

OUTPUT · 10 ROWS

customer_id	document_type	verified_by
1	Aadhaar	Rahul Sharma
2	PAN	Aman Khan
3	Aadhaar	Rohit Verma
4	Passport	Arjun Mehta
5	PAN	Nisha Singh
6	Aadhaar	Karan Rao
7	PAN	Not Assigned
8	Aadhaar	Arjun Mehta
9	Passport	Karan Rao
10	PAN	Not Assigned

Q7 Display 'No Remarks' when KYC remarks are NULL.

```
SELECT
    customer_id,
    verification_status,
    COALESCE(remarks, 'No Remarks') AS remarks
FROM KYC;
```

OUTPUT · 10 ROWS

customer_id	verification_status	remarks
1	Verified	No Remarks
2	Verified	No Remarks
3	Verified	No Remarks
4	Verified	No Remarks
5	Verified	No Remarks
6	Verified	No Remarks
7	Pending	No Remarks
8	Verified	No Remarks
9	Verified	No Remarks
10	Pending	No Remarks

44 Inner Join

PART III

Q1 Find the account number and balance of each customer.

```sql
SELECT
    a.account_id,
    a.account_number,
    c.first_name,
    c.last_name,
    a.balance
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id;
```

OUTPUT · 10 ROWS

account_id	account_number	first_name	last_name	balance
1	ACC100001	Aarav	Sharma	75000.00
2	ACC100002	Ananya	Patel	45000.00
3	ACC100003	Rohan	Verma	150000.00
4	ACC100004	Simran	Khan	60000.00
5	ACC100005	Aditya	Mehta	85000.00
6	ACC100006	Isha	Gupta	55000.00
7	ACC100007	Kabir	Joshi	120000.00
8	ACC100008	Meera	Singh	35000.00
9	ACC100009	Vivek	Rao	200000.00
10	ACC100010	Diya	Kapoor	95000.00

Q2 Display the customer name, account number, branch name, and balance for each account.

```sql
SELECT
    a.account_number,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    b.branch_name,
    a.balance
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
INNER JOIN Branches b
    ON a.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

account_number	customer_name	branch_name	balance
ACC100001	Aarav Sharma	Main Branch	75000.00
ACC100002	Ananya Patel	Andheri Branch	45000.00
ACC100003	Rohan Verma	Bandra Branch	150000.00
ACC100004	Simran Khan	Pune Branch	60000.00
ACC100005	Aditya Mehta	Delhi Branch	85000.00
ACC100006	Isha Gupta	Bangalore Branch	55000.00
ACC100007	Kabir Joshi	Main Branch	120000.00
ACC100008	Meera Singh	Pune Branch	35000.00
ACC100009	Vivek Rao	Bangalore Branch	200000.00
ACC100010	Diya Kapoor	Delhi Branch	95000.00

Q3 **Find the employee name, job title, and branch name for each employee.**

```
SELECT
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    e.job_title,
    b.branch_name
FROM Employees e
INNER JOIN Branches b
    ON e.branch_id = b.branch_id;
```

OUTPUT · 9 ROWS

employee_name	job_title	branch_name
Rahul Sharma	Branch Manager	Main Branch
Sneha Patel	Cashier	Main Branch
Aman Khan	Branch Manager	Andheri Branch
Pooja Gupta	Loan Officer	Andheri Branch
Rohit Verma	Branch Manager	Bandra Branch
Kavita Joshi	Customer Service	Bandra Branch
Arjun Mehta	Branch Manager	Pune Branch
Nisha Singh	Loan Officer	Delhi Branch
Karan Rao	Cashier	Bangalore Branch

Q4 **Find the account number, customer name, account type, and interest rate for each account.**

```
SELECT
    a.account_number,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    at.account_type_name,
    at.interest_rate
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
INNER JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id;
```

OUTPUT · 10 ROWS

account_number	customer_name	account_type_name	interest_rate
ACC100001	Aarav Sharma	Savings	4.00
ACC100002	Ananya Patel	Savings	4.00
ACC100003	Rohan Verma	Current	0.00
ACC100004	Simran Khan	Savings	4.00
ACC100005	Aditya Mehta	Salary	3.50
ACC100006	Isha Gupta	Savings	4.00
ACC100007	Kabir Joshi	Senior Citizen Savings	5.50
ACC100008	Meera Singh	Savings	4.00
ACC100009	Vivek Rao	Current	0.00
ACC100010	Diya Kapoor	NRI Savings	4.50

Q5 **Find the account details of customers who live in Mumbai.**

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE c.city = 'Mumbai';
```

OUTPUT · 4 ROWS

customer_name	account_number	balance
Aarav Sharma	ACC100001	75000.00
Ananya Patel	ACC100002	45000.00
Rohan Verma	ACC100003	150000.00
Kabir Joshi	ACC100007	120000.00

Q6 **Find the name, account number, and balance of all Savings account holders.**

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
INNER JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id
WHERE at.account_type_name = 'Savings';
```

OUTPUT · 5 ROWS

customer_name	account_number	balance
Aarav Sharma	ACC100001	75000.00
Ananya Patel	ACC100002	45000.00
Simran Khan	ACC100004	60000.00
Isha Gupta	ACC100006	55000.00
Meera Singh	ACC100008	35000.00

Q7 **Display the loan details along with the customer name and branch name.**

```
SELECT
    l.loan_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    b.branch_name,
    l.loan_type,
    l.loan_amount
FROM Loans l
INNER JOIN Customers c
    ON l.customer_id = c.customer_id
INNER JOIN Branches b
    ON l.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

loan_id	customer_name	branch_name	loan_type	loan_amount
1	Aarav Sharma	Main Branch	Home Loan	2500000.00
2	Ananya Patel	Andheri Branch	Personal Loan	500000.00
3	Rohan Verma	Bandra Branch	Business Loan	5000000.00

loan_id	customer_name	branch_name	loan_type	loan_amount
4	Simran Khan	Pune Branch	Education Loan	800000.00
5	Aditya Mehta	Delhi Branch	Car Loan	1200000.00
6	Isha Gupta	Bangalore Branch	Personal Loan	300000.00
7	Kabir Joshi	Main Branch	Home Loan	3500000.00
8	Meera Singh	Pune Branch	Education Loan	600000.00
9	Vivek Rao	Bangalore Branch	Business Loan	4000000.00
10	Diya Kapoor	Delhi Branch	Car Loan	900000.00

Q8 **Display each complaint with the customer name and assigned employee name.**

```
SELECT
    co.complaint_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    co.complaint_type,
    co.status
FROM Complaints co
INNER JOIN Customers c
    ON co.customer_id = c.customer_id
INNER JOIN Employees e
    ON co.assigned_employee_id = e.employee_id;
```

OUTPUT · 10 ROWS

complaint_id	customer_name	employee_name	complaint_type	status
1	Aarav Sharma	Sneha Patel	ATM Issue	Resolved
2	Ananya Patel	Aman Khan	Transaction Issue	In Progress
3	Rohan Verma	Rohit Verma	Card Issue	Open
4	Simran Khan	Arjun Mehta	Account Issue	Resolved
5	Aditya Mehta	Nisha Singh	Loan Issue	In Progress
6	Isha Gupta	Karan Rao	Internet Banking	Open
7	Kabir Joshi	Rahul Sharma	Fund Transfer	Resolved
8	Meera Singh	Arjun Mehta	ATM Issue	Open
9	Vivek Rao	Karan Rao	Loan Issue	In Progress
10	Diya Kapoor	Nisha Singh	Card Issue	Resolved

45 Left Join

PART III

Q1 Display all customers and their account numbers.

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance
FROM Customers c
LEFT JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

OUTPUT · 10 ROWS

customer_id	customer_name	account_number	balance
1	Aarav Sharma	ACC100001	75000.00
2	Ananya Patel	ACC100002	45000.00
3	Rohan Verma	ACC100003	150000.00
4	Simran Khan	ACC100004	60000.00
5	Aditya Mehta	ACC100005	85000.00
6	Isha Gupta	ACC100006	55000.00
7	Kabir Joshi	ACC100007	120000.00
8	Meera Singh	ACC100008	35000.00
9	Vivek Rao	ACC100009	200000.00
10	Diya Kapoor	ACC100010	95000.00

Q2 Display all branches and the number of accounts opened in each branch.

```
SELECT
    b.branch_id,
    b.branch_name,
    COUNT(a.account_id) AS total_accounts
FROM Branches b
LEFT JOIN Accounts a
    ON b.branch_id = a.branch_id
GROUP BY b.branch_id, b.branch_name;
```

OUTPUT · 6 ROWS

branch_id	branch_name	total_accounts
1	Main Branch	2
2	Andheri Branch	1
3	Bandra Branch	1
4	Pune Branch	2
5	Delhi Branch	2
6	Bangalore Branch	2

Q3 **Display all customers and their loan details, including customers who do not have any loan.**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    l.loan_type,
    l.loan_amount
FROM Customers c
LEFT JOIN Loans l
    ON c.customer_id = l.customer_id;
```

OUTPUT · 10 ROWS

customer_id	customer_name	loan_type	loan_amount
1	Aarav Sharma	Home Loan	2500000.00
2	Ananya Patel	Personal Loan	500000.00
3	Rohan Verma	Business Loan	5000000.00
4	Simran Khan	Education Loan	800000.00
5	Aditya Mehta	Car Loan	1200000.00
6	Isha Gupta	Personal Loan	300000.00
7	Kabir Joshi	Home Loan	3500000.00
8	Meera Singh	Education Loan	600000.00
9	Vivek Rao	Business Loan	4000000.00
10	Diya Kapoor	Car Loan	900000.00

Q4 **Find customers who do not have any loan.**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name
FROM Customers c
LEFT JOIN Loans l
    ON c.customer_id = l.customer_id
WHERE l.loan_id IS NULL;
```

OUTPUT · 0 ROWS

customer_id	customer_name

Empty set: no rows match this condition in the sample data.

Q5 **Display all customers and their KYC verification status, including customers without KYC records.**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    k.verification_status
FROM Customers c
LEFT JOIN KYC k
    ON c.customer_id = k.customer_id;
```

OUTPUT · 10 ROWS

customer_id	customer_name	verification_status
1	Aarav Sharma	Verified
2	Ananya Patel	Verified
3	Rohan Verma	Verified
4	Simran Khan	Verified

customer_id	customer_name	verification_status
5	Aditya Mehta	Verified
6	Isha Gupta	Verified
7	Kabir Joshi	Pending
8	Meera Singh	Verified
9	Vivek Rao	Verified
10	Diya Kapoor	Pending

Q6 **Find branches that currently have no employees.**

```
SELECT
    b.branch_id,
    b.branch_name
FROM Branches b
LEFT JOIN Employees e
    ON b.branch_id = e.branch_id
WHERE e.employee_id IS NULL;
```

OUTPUT · 0 ROWS

branch_id	branch_name

Empty set: no rows match this condition in the sample data.

Q7 **Display all customers with their internet banking username, including customers who have not registered for internet banking.**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    ib.username
FROM Customers c
LEFT JOIN InternetBanking ib
    ON c.customer_id = ib.customer_id;
```

OUTPUT · 10 ROWS

customer_id	customer_name	username
1	Aarav Sharma	aarav123
2	Ananya Patel	ananya123
3	Rohan Verma	rohan123
4	Simran Khan	simran123
5	Aditya Mehta	aditya123
6	Isha Gupta	isha123
7	Kabir Joshi	kabir123
8	Meera Singh	meera123
9	Vivek Rao	vivek123
10	Diya Kapoor	diya123

Q8 **Find customers who have not registered for internet banking.**

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name
FROM Customers c
LEFT JOIN InternetBanking ib
    ON c.customer_id = ib.customer_id
WHERE ib.internet_banking_id IS NULL;
```

OUTPUT · 0 ROWS

customer_id	customer_name

Empty set: no rows match this condition in the sample data.

46 Right Join

PART III

Q1 **Display all branches and their employees.**

```
SELECT
    b.branch_name,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    e.job_title
FROM Employees e
RIGHT JOIN Branches b
    ON e.branch_id = b.branch_id;
```

OUTPUT · 9 ROWS

branch_name	employee_name	job_title
Main Branch	Rahul Sharma	Branch Manager
Main Branch	Sneha Patel	Cashier
Andheri Branch	Aman Khan	Branch Manager
Andheri Branch	Pooja Gupta	Loan Officer
Bandra Branch	Rohit Verma	Branch Manager
Bandra Branch	Kavita Joshi	Customer Service
Pune Branch	Arjun Mehta	Branch Manager
Delhi Branch	Nisha Singh	Loan Officer
Bangalore Branch	Karan Rao	Cashier

Q2 Display all branches along with their account numbers and balances.

```
SELECT
    b.branch_name,
    a.account_number,
    a.balance
FROM Accounts a
RIGHT JOIN Branches b
    ON a.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

branch_name	account_number	balance
Main Branch	ACC100001	75000.00
Andheri Branch	ACC100002	45000.00
Bandra Branch	ACC100003	150000.00
Pune Branch	ACC100004	60000.00
Delhi Branch	ACC100005	85000.00
Bangalore Branch	ACC100006	55000.00
Main Branch	ACC100007	120000.00
Pune Branch	ACC100008	35000.00
Bangalore Branch	ACC100009	200000.00
Delhi Branch	ACC100010	95000.00

Q3 Display all account types and the accounts belonging to each type.

```
SELECT
    at.account_type_name,
    a.account_number,
    a.balance
FROM Accounts a
RIGHT JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id;
```

OUTPUT · 10 ROWS

account_type_name	account_number	balance
Savings	ACC100001	75000.00
Savings	ACC100002	45000.00
Current	ACC100003	150000.00
Savings	ACC100004	60000.00
Salary	ACC100005	85000.00
Savings	ACC100006	55000.00
Senior Citizen Savings	ACC100007	120000.00
Savings	ACC100008	35000.00
Current	ACC100009	200000.00
NRI Savings	ACC100010	95000.00

Q4 Find account types that currently have no accounts.

```sql
SELECT
    at.account_type_id,
    at.account_type_name
FROM Accounts a
RIGHT JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id
WHERE a.account_id IS NULL;
```

OUTPUT · 0 ROWS

account_type_id	account_type_name

Empty set: no rows match this condition in the sample data.

Q5 Display all branches and the number of employees working in each branch.

```sql
SELECT
    b.branch_id,
    b.branch_name,
    COUNT(e.employee_id) AS total_employees
FROM Employees e
RIGHT JOIN Branches b
    ON e.branch_id = b.branch_id
GROUP BY b.branch_id, b.branch_name;
```

OUTPUT · 6 ROWS

branch_id	branch_name	total_employees
1	Main Branch	2
2	Andheri Branch	2
3	Bandra Branch	2
4	Pune Branch	1
5	Delhi Branch	1
6	Bangalore Branch	1

Q6 Display all account types along with the average balance of their accounts.

```sql
SELECT
    at.account_type_name,
    AVG(a.balance) AS average_balance
FROM Accounts a
RIGHT JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id
GROUP BY at.account_type_id, at.account_type_name;
```

OUTPUT · 5 ROWS

account_type_name	average_balance
Savings	54000.000000
Current	175000.000000
Salary	85000.000000
Senior Citizen Savings	120000.000000
NRI Savings	95000.000000

47 Self Join

PART III

Q1 Find pairs of employees who work in the same branch.

```sql
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS employee_1,
    CONCAT(e2.first_name, ' ', e2.last_name) AS employee_2,
    e1.branch_id
FROM Employees e1
INNER JOIN Employees e2
    ON e1.branch_id = e2.branch_id
   AND e1.employee_id < e2.employee_id;
```

OUTPUT · 3 ROWS

employee_1	employee_2	branch_id
Rahul Sharma	Sneha Patel	1
Aman Khan	Pooja Gupta	2
Rohit Verma	Kavita Joshi	3

Q2 Find employees who have the same job title.

```sql
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS employee_1,
    CONCAT(e2.first_name, ' ', e2.last_name) AS employee_2,
    e1.job_title
FROM Employees e1
INNER JOIN Employees e2
    ON e1.job_title = e2.job_title
   AND e1.employee_id < e2.employee_id;
```

OUTPUT · 8 ROWS

employee_1	employee_2	job_title
Rahul Sharma	Aman Khan	Branch Manager
Rahul Sharma	Arjun Mehta	Branch Manager
Rahul Sharma	Rohit Verma	Branch Manager
Sneha Patel	Karan Rao	Cashier
Aman Khan	Arjun Mehta	Branch Manager
Aman Khan	Rohit Verma	Branch Manager
Pooja Gupta	Nisha Singh	Loan Officer
Rohit Verma	Arjun Mehta	Branch Manager

Q3 Find employees working in the same branch with different salaries.

```
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS employee_1,
    e1.salary AS salary_1,
    CONCAT(e2.first_name, ' ', e2.last_name) AS employee_2,
    e2.salary AS salary_2,
    e1.branch_id
FROM Employees e1
INNER JOIN Employees e2
    ON e1.branch_id = e2.branch_id
   AND e1.salary <> e2.salary
   AND e1.employee_id < e2.employee_id;
```

OUTPUT · 3 ROWS

employee_1	salary_1	employee_2	salary_2	branch_id
Rahul Sharma	85000	Sneha Patel	45000	1
Aman Khan	82000	Pooja Gupta	60000	2
Rohit Verma	80000	Kavita Joshi	48000	3

Q4 Find employees whose salary is higher than another employee in the same branch.

```
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS higher_paid_employee,
    e1.salary AS higher_salary,
    CONCAT(e2.first_name, ' ', e2.last_name) AS lower_paid_employee,
    e2.salary AS lower_salary,
    e1.branch_id
FROM Employees e1
INNER JOIN Employees e2
    ON e1.branch_id = e2.branch_id
   AND e1.salary > e2.salary;
```

OUTPUT · 3 ROWS

higher_paid_employee	higher_salary	lower_paid_employee	lower_salary	branch_id
Rahul Sharma	85000	Sneha Patel	45000	1
Aman Khan	82000	Pooja Gupta	60000	2
Rohit Verma	80000	Kavita Joshi	48000	3

Q5 Find employees who joined later than another employee in the same branch.

```
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS later_employee,
    e1.hire_date AS later_hire_date,
    CONCAT(e2.first_name, ' ', e2.last_name) AS earlier_employee,
    e2.hire_date AS earlier_hire_date,
    e1.branch_id
FROM Employees e1
INNER JOIN Employees e2
    ON e1.branch_id = e2.branch_id
   AND e1.hire_date > e2.hire_date;
```

OUTPUT · 3 ROWS

later_employee	later_hire_date	earlier_employee	earlier_hire_date	branch_id
Sneha Patel	2020-07-15	Rahul Sharma	2018-04-10	1
Pooja Gupta	2021-05-12	Aman Khan	2019-02-20	2
Kavita Joshi	2022-01-10	Rohit Verma	2019-08-18	3

48 Multi-Table JOIN

PART III

Q1 Display each customer's name, account number, account type, branch name, and balance.

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    at.account_type_name,
    b.branch_name,
    a.balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id
INNER JOIN Branches b
    ON a.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

customer_name	account_number	account_type_name	branch_name	balance
Aarav Sharma	ACC100001	Savings	Main Branch	75000.00
Ananya Patel	ACC100002	Savings	Andheri Branch	45000.00
Rohan Verma	ACC100003	Current	Bandra Branch	150000.00
Simran Khan	ACC100004	Savings	Pune Branch	60000.00
Aditya Mehta	ACC100005	Salary	Delhi Branch	85000.00
Isha Gupta	ACC100006	Savings	Bangalore Branch	55000.00
Kabir Joshi	ACC100007	Senior Citizen Savings	Main Branch	120000.00
Meera Singh	ACC100008	Savings	Pune Branch	35000.00
Vivek Rao	ACC100009	Current	Bangalore Branch	200000.00
Diya Kapoor	ACC100010	NRI Savings	Delhi Branch	95000.00

Q2 Display each loan with customer name, branch name, loan type, interest rate, and loan amount.

```
SELECT
    l.loan_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    b.branch_name,
    l.loan_type,
    l.interest_rate,
    l.loan_amount
FROM Loans l
INNER JOIN Customers c
    ON l.customer_id = c.customer_id
INNER JOIN Branches b
    ON l.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

loan_id	customer_name	branch_name	loan_type	interest_rate	loan_amount
1	Aarav Sharma	Main Branch	Home Loan	8.50	2500000.00
2	Ananya Patel	Andheri Branch	Personal Loan	11.00	500000.00
3	Rohan Verma	Bandra Branch	Business Loan	9.25	5000000.00
4	Simran Khan	Pune Branch	Education Loan	7.50	800000.00
5	Aditya Mehta	Delhi Branch	Car Loan	9.00	1200000.00
6	Isha Gupta	Bangalore Branch	Personal Loan	10.50	300000.00
7	Kabir Joshi	Main Branch	Home Loan	8.25	3500000.00

loan_id	customer_name	branch_name	loan_type	interest_rate	loan_amount
8	Meera Singh	Pune Branch	Education Loan	7.25	600000.00
9	Vivek Rao	Bangalore Branch	Business Loan	9.50	4000000.00
10	Diya Kapoor	Delhi Branch	Car Loan	8.75	900000.00

Q3 **Display each loan payment with customer name, loan type, payment amount, and payment date.**

```
SELECT
    lp.payment_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    l.loan_type,
    lp.payment_amount,
    lp.payment_date
FROM LoanPayments lp
INNER JOIN Loans l
    ON lp.loan_id = l.loan_id
INNER JOIN Customers c
    ON l.customer_id = c.customer_id;
```

OUTPUT · 11 ROWS

payment_id	customer_name	loan_type	payment_amount	payment_date
1	Aarav Sharma	Home Loan	25000.00	2026-10-29
2	Aarav Sharma	Home Loan	25000.00	2026-10-29
3	Ananya Patel	Personal Loan	11000.00	2026-10-29
4	Rohan Verma	Business Loan	65000.00	2026-10-29
5	Simran Khan	Education Loan	12000.00	2026-10-29
6	Aditya Mehta	Car Loan	25000.00	2026-10-29
7	Isha Gupta	Personal Loan	10000.00	2026-10-29
8	Kabir Joshi	Home Loan	35000.00	2026-10-29
9	Meera Singh	Education Loan	12000.00	2026-10-29
10	Vivek Rao	Business Loan	55000.00	2026-10-29
11	Diya Kapoor	Car Loan	19000.00	2026-10-29

48B Insert Fund Transfer Data

PART III

SQL Insert Fund Transfer Data

```sql
INSERT INTO FundTransfers
(sender_account_id, receiver_account_id, amount, transfer_type, reference_number, remarks)
VALUES
(1, 2, 5000.00, 'NEFT', 'FT100001', 'Monthly transfer'),
(2, 3, 7500.00, 'IMPS', 'FT100002', 'Personal transfer'),
(3, 4, 10000.00, 'RTGS', 'FT100003', 'Business payment'),
(4, 5, 4500.00, 'UPI', 'FT100004', 'Online payment'),
(5, 6, 12000.00, 'NEFT', 'FT100005', 'Salary transfer'),
(6, 7, 6500.00, 'IMPS', 'FT100006', 'Personal transfer'),
(7, 8, 9000.00, 'UPI', 'FT100007', 'Bill payment'),
(8, 9, 15000.00, 'RTGS', 'FT100008', 'Business transfer'),
(9, 10, 8000.00, 'NEFT', 'FT100009', 'Family transfer'),
(10, 1, 5500.00, 'UPI', 'FT100010', 'Personal transfer');
```

RESULT · 10 ROWS INSERTED

SQL Insert Fund Transfer Data

```sql
SELECT * FROM FundTransfers;
```

OUTPUT · 10 ROWS

transfer_id	sender_account_id	receiver_account_id	amount	transfer_type	transfer_date	reference_number	remarks	status
1	1	2	5000.00	NEFT	2026-10-29 07:09:08	FT100001	Monthly transfer	Success
2	2	3	7500.00	IMPS	2026-10-29 07:09:08	FT100002	Personal transfer	Success
3	3	4	10000.00	RTGS	2026-10-29 07:09:08	FT100003	Business payment	Success
4	4	5	4500.00	UPI	2026-10-29 07:09:08	FT100004	Online payment	Success
5	5	6	12000.00	NEFT	2026-10-29 07:09:08	FT100005	Salary transfer	Success
6	6	7	6500.00	IMPS	2026-10-29 07:09:08	FT100006	Personal transfer	Success
7	7	8	9000.00	UPI	2026-10-29 07:09:08	FT100007	Bill payment	Success
8	8	9	15000.00	RTGS	2026-10-29 07:09:08	FT100008	Business transfer	Success
9	9	10	8000.00	NEFT	2026-10-29 07:09:08	FT100009	Family transfer	Success
10	10	1	5500.00	UPI	2026-10-29 07:09:08	FT100010	Personal transfer	Success

Q4 Display each fund transfer with sender account number, receiver account number, and transfer amount.

```sql
SELECT
    ft.transfer_id,
    a1.account_number AS sender_account,
    a2.account_number AS receiver_account,
    ft.amount,
    ft.transfer_type,
    ft.transfer_date
FROM FundTransfers ft
JOIN Accounts a1
    ON ft.sender_account_id = a1.account_id
JOIN Accounts a2
    ON ft.receiver_account_id = a2.account_id;
```

OUTPUT · 10 ROWS

transfer_id	sender_account	receiver_account	amount	transfer_type	transfer_date
1	ACC100001	ACC100002	5000.00	NEFT	2026-10-29 07:09:08
2	ACC100002	ACC100003	7500.00	IMPS	2026-10-29 07:09:08
3	ACC100003	ACC100004	10000.00	RTGS	2026-10-29 07:09:08
4	ACC100004	ACC100005	4500.00	UPI	2026-10-29 07:09:08
5	ACC100005	ACC100006	12000.00	NEFT	2026-10-29 07:09:08
6	ACC100006	ACC100007	6500.00	IMPS	2026-10-29 07:09:08
7	ACC100007	ACC100008	9000.00	UPI	2026-10-29 07:09:08
8	ACC100008	ACC100009	15000.00	RTGS	2026-10-29 07:09:08
9	ACC100009	ACC100010	8000.00	NEFT	2026-10-29 07:09:08
10	ACC100010	ACC100001	5500.00	UPI	2026-10-29 07:09:08

SQL Insert Fund Transfer Data

```sql
INSERT INTO ATMTransactions
(account_id, card_id, transaction_type, amount, atm_location)
VALUES
(1, 1, 'Withdrawal', 5000.00, 'Mumbai Main Branch ATM'),
(2, 2, 'Withdrawal', 3000.00, 'Andheri ATM'),
(3, 3, 'Withdrawal', 10000.00, 'Bandra ATM'),
(4, 4, 'Deposit', 8000.00, 'Pune FC Road ATM'),
(5, 5, 'Withdrawal', 7000.00, 'Delhi Central ATM'),
(6, 6, 'Withdrawal', 4000.00, 'Bangalore MG Road ATM'),
(7, 7, 'Withdrawal', 6000.00, 'Mumbai Main Branch ATM'),
(8, 8, 'Deposit', 5000.00, 'Pune Viman Nagar ATM'),
(9, 9, 'Withdrawal', 12000.00, 'Bangalore Indiranagar ATM'),
(10, 10, 'Withdrawal', 3500.00, 'Delhi Gurgaon Road ATM');
```

SQL **Insert Fund Transfer Data**

```
SELECT * FROM ATMTransactions;
```

OUTPUT · 10 ROWS

atm_transaction_id	account_id	card_id	transaction_type	amount	atm_location	transaction_date	status
1	1	1	Withdrawal	5000.00	Mumbai Main Branch ATM	2026-10-29 07:09:08	Success
2	2	2	Withdrawal	3000.00	Andheri ATM	2026-10-29 07:09:08	Success
3	3	3	Withdrawal	10000.00	Bandra ATM	2026-10-29 07:09:08	Success
4	4	4	Deposit	8000.00	Pune FC Road ATM	2026-10-29 07:09:08	Success
5	5	5	Withdrawal	7000.00	Delhi Central ATM	2026-10-29 07:09:08	Success
6	6	6	Withdrawal	4000.00	Bangalore MG Road ATM	2026-10-29 07:09:08	Success
7	7	7	Withdrawal	6000.00	Mumbai Main Branch ATM	2026-10-29 07:09:08	Success
8	8	8	Deposit	5000.00	Pune Viman Nagar ATM	2026-10-29 07:09:08	Success
9	9	9	Withdrawal	12000.00	Bangalore Indiranagar ATM	2026-10-29 07:09:08	Success
10	10	10	Withdrawal	3500.00	Delhi Gurgaon Road ATM	2026-10-29 07:09:08	Success

Q5 **Display each ATM transaction with customer name, account number, card number, transaction type, and amount.**

```
SELECT
    atm.atm_transaction_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    cd.card_number,
    atm.transaction_type,
    atm.amount
FROM ATMTransactions atm
INNER JOIN Accounts a
    ON atm.account_id = a.account_id
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
INNER JOIN Cards cd
    ON atm.card_id = cd.card_id;
```

OUTPUT · 10 ROWS

atm_transaction_id	customer_name	account_number	card_number	transaction_type	amount
1	Aarav Sharma	ACC100001	4500123456780001	Withdrawal	5000.00
2	Ananya Patel	ACC100002	4500123456780002	Withdrawal	3000.00
3	Rohan Verma	ACC100003	4500123456780003	Withdrawal	10000.00
4	Simran Khan	ACC100004	4500123456780004	Deposit	8000.00
5	Aditya Mehta	ACC100005	4500123456780005	Withdrawal	7000.00
6	Isha Gupta	ACC100006	4500123456780006	Withdrawal	4000.00
7	Kabir Joshi	ACC100007	4500123456780007	Withdrawal	6000.00
8	Meera Singh	ACC100008	4500123456780008	Deposit	5000.00
9	Vivek Rao	ACC100009	4500123456780009	Withdrawal	12000.00
10	Diya Kapoor	ACC100010	4500123456780010	Withdrawal	3500.00

Q6 **Display each complaint with customer name, employee name, branch name, complaint type, and status.**

```
SELECT
    co.complaint_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    b.branch_name,
    co.complaint_type,
    co.status
FROM Complaints co
INNER JOIN Customers c
    ON co.customer_id = c.customer_id
INNER JOIN Employees e
    ON co.assigned_employee_id = e.employee_id
INNER JOIN Branches b
    ON e.branch_id = b.branch_id;
```

OUTPUT · 10 ROWS

complaint_id	customer_name	employee_name	branch_name	complaint_type	status
1	Aarav Sharma	Sneha Patel	Main Branch	ATM Issue	Resolved
2	Ananya Patel	Aman Khan	Andheri Branch	Transaction Issue	In Progress
3	Rohan Verma	Rohit Verma	Bandra Branch	Card Issue	Open
4	Simran Khan	Arjun Mehta	Pune Branch	Account Issue	Resolved
5	Aditya Mehta	Nisha Singh	Delhi Branch	Loan Issue	In Progress
6	Isha Gupta	Karan Rao	Bangalore Branch	Internet Banking	Open
7	Kabir Joshi	Rahul Sharma	Main Branch	Fund Transfer	Resolved
8	Meera Singh	Arjun Mehta	Pune Branch	ATM Issue	Open
9	Vivek Rao	Karan Rao	Bangalore Branch	Loan Issue	In Progress
10	Diya Kapoor	Nisha Singh	Delhi Branch	Card Issue	Resolved

Q7 **Display each fixed deposit with customer name, account number, principal amount, interest rate, and maturity amount.**

```
SELECT
    fd.fd_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    fd.principal_amount,
    fd.interest_rate,
    fd.maturity_amount
FROM FixedDeposits fd
INNER JOIN Customers c
    ON fd.customer_id = c.customer_id
INNER JOIN Accounts a
    ON fd.account_id = a.account_id;
```

OUTPUT · 10 ROWS

fd_id	customer_name	account_number	principal_amount	interest_rate	maturity_amount
1	Aarav Sharma	ACC100001	100000.00	6.50	106500.00
2	Ananya Patel	ACC100002	150000.00	6.75	170250.00
3	Rohan Verma	ACC100003	500000.00	7.00	605000.00
4	Simran Khan	ACC100004	200000.00	6.50	213000.00
5	Aditya Mehta	ACC100005	300000.00	7.25	343500.00
6	Isha Gupta	ACC100006	100000.00	6.50	106500.00
7	Kabir Joshi	ACC100007	400000.00	7.50	490000.00

fd_id	customer_name	account_number	principal_amount	interest_rate	maturity_amount
8	Meera Singh	ACC100008	175000.00	6.75	198625.00
9	Vivek Rao	ACC100009	600000.00	7.25	730500.00
10	Diya Kapoor	ACC100010	250000.00	7.00	267500.00

Q8 **Display each recurring deposit with customer name, account number, monthly amount, interest rate, and maturity amount.**

```
SELECT
    rd.rd_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    rd.monthly_amount,
    rd.interest_rate,
    rd.maturity_amount
FROM RecurringDeposits rd
INNER JOIN Customers c
    ON rd.customer_id = c.customer_id
INNER JOIN Accounts a
    ON rd.account_id = a.account_id;
```

OUTPUT · 10 ROWS

rd_id	customer_name	account_number	monthly_amount	interest_rate	maturity_amount
1	Aarav Sharma	ACC100001	5000.00	6.50	62000.00
2	Ananya Patel	ACC100002	7500.00	6.75	190000.00
3	Rohan Verma	ACC100003	10000.00	7.00	390000.00
4	Simran Khan	ACC100004	4000.00	6.50	49600.00
5	Aditya Mehta	ACC100005	8000.00	7.25	210000.00
6	Isha Gupta	ACC100006	5000.00	6.50	62000.00
7	Kabir Joshi	ACC100007	12000.00	7.50	480000.00
8	Meera Singh	ACC100008	6000.00	6.75	152000.00
9	Vivek Rao	ACC100009	15000.00	7.25	585000.00
10	Diya Kapoor	ACC100010	7000.00	7.00	86800.00

49 Subqueries PART III

Q1 Find customers whose account balance is greater than the average account balance.

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE a.balance > (
    SELECT AVG(balance)
    FROM Accounts
);
```

OUTPUT · 4 ROWS

customer_id	customer_name	account_number	balance
3	Rohan Verma	ACC100003	150000.00
7	Kabir Joshi	ACC100007	120000.00
9	Vivek Rao	ACC100009	200000.00
10	Diya Kapoor	ACC100010	95000.00

Q2 Find Customers with the Highest Account Balance.

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE a.balance = (
    SELECT MAX(balance)
    FROM Accounts
);
```

OUTPUT · 1 ROW

customer_id	customer_name	account_number	balance
9	Vivek Rao	ACC100009	200000.00

Q3 Find Accounts with Balance Above the Average.

```sql
SELECT
    account_id,
    account_number,
    balance
FROM Accounts
WHERE balance > (
    SELECT AVG(balance)
    FROM Accounts
);
```

OUTPUT · 4 ROWS

account_id	account_number	balance
3	ACC100003	150000.00
7	ACC100007	120000.00
9	ACC100009	200000.00
10	ACC100010	95000.00

Q4 Find Customers Who Have Taken a Loan.

```sql
SELECT
    customer_id,
    CONCAT(first_name, ' ', last_name) AS customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Loans
);
```

OUTPUT · 10 ROWS

customer_id	customer_name
1	Aarav Sharma
2	Ananya Patel
3	Rohan Verma
4	Simran Khan
5	Aditya Mehta
6	Isha Gupta
7	Kabir Joshi
8	Meera Singh
9	Vivek Rao
10	Diya Kapoor

Q5 Find Customers Who Have Not Taken Any Loan.

```sql
SELECT
    customer_id,
    CONCAT(first_name, ' ', last_name) AS customer_name
FROM Customers
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM Loans
);
```

OUTPUT · 0 ROWS

customer_id	customer_name

Empty set: no rows match this condition in the sample data.

Q6 Find Customers Who Have Made Transactions.

```sql
SELECT
    customer_id,
    CONCAT(first_name, ' ', last_name) AS customer_name
FROM Customers
WHERE customer_id IN (
    SELECT DISTINCT a.customer_id
    FROM Accounts a
    INNER JOIN Transactions t
        ON a.account_id = t.account_id
);
```

OUTPUT · 10 ROWS

customer_id	customer_name
1	Aarav Sharma
2	Ananya Patel
3	Rohan Verma
4	Simran Khan
5	Aditya Mehta
6	Isha Gupta
7	Kabir Joshi
8	Meera Singh
9	Vivek Rao
10	Diya Kapoor

Q7 Find Customers with an Account Balance Above Their Branch Average

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.branch_id,
    a.balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
WHERE a.balance > (
    SELECT AVG(a2.balance)
    FROM Accounts a2
    WHERE a2.branch_id = a.branch_id
);
```

OUTPUT · 4 ROWS

customer_id	customer_name	account_number	branch_id	balance
4	Simran Khan	ACC100004	4	60000.00
7	Kabir Joshi	ACC100007	1	120000.00
9	Vivek Rao	ACC100009	6	200000.00
10	Diya Kapoor	ACC100010	5	95000.00

Q8 Find Accounts with Balance Greater Than All Savings Accounts

```sql
SELECT
    a.account_id,
    a.account_number,
    a.balance
FROM Accounts a
WHERE a.balance > ALL (
    SELECT a2.balance
    FROM Accounts a2
    INNER JOIN AccountTypes at
        ON a2.account_type_id = at.account_type_id
    WHERE at.account_type_name = 'Savings'
);
```

OUTPUT · 5 ROWS

account_id	account_number	balance
3	ACC100003	150000.00
5	ACC100005	85000.00
7	ACC100007	120000.00
9	ACC100009	200000.00
10	ACC100010	95000.00

50 Common Table Expressions (CTE) PART III

Q1 Calculate Total Transaction Amount for Each Account

```sql
WITH AccountTransactions AS (
    SELECT
        account_id,
        SUM(amount) AS total_transaction_amount
    FROM Transactions
    GROUP BY account_id
)
SELECT
    a.account_number,
    at.total_transaction_amount
FROM AccountTransactions at
INNER JOIN Accounts a
    ON at.account_id = a.account_id;
```

OUTPUT · 10 ROWS

account_number	total_transaction_amount
ACC100001	30000.00
ACC100002	18000.00
ACC100003	60000.00
ACC100004	20000.00
ACC100005	30000.00
ACC100006	7000.00
ACC100007	40000.00
ACC100008	5000.00
ACC100009	75000.00
ACC100010	15000.00

Q2 Find Accounts with Total Transactions Greater Than 30,000

```sql
WITH AccountTransactions AS (
    SELECT
        account_id,
        SUM(amount) AS total_transaction_amount
    FROM Transactions
    GROUP BY account_id
)
SELECT
    a.account_number,
    at.total_transaction_amount
FROM AccountTransactions at
INNER JOIN Accounts a
    ON at.account_id = a.account_id
WHERE at.total_transaction_amount > 30000;
```

OUTPUT · 3 ROWS

account_number	total_transaction_amount
ACC100003	60000.00
ACC100007	40000.00
ACC100009	75000.00

Q3 Find the Average Loan Amount by Loan Type

```sql
WITH LoanSummary AS (
    SELECT
        loan_type,
        AVG(loan_amount) AS average_loan_amount
    FROM Loans
    GROUP BY loan_type
)
SELECT
    loan_type,
    ROUND(average_loan_amount, 2) AS average_loan_amount
FROM LoanSummary
ORDER BY average_loan_amount DESC;
```

OUTPUT · 5 ROWS

loan_type	average_loan_amount
Business Loan	4500000.000000
Home Loan	3000000.000000
Car Loan	1050000.000000
Education Loan	700000.000000
Personal Loan	400000.000000

Q4 Find Branches with More Than One Employee

```sql
WITH BranchEmployeeCount AS (
    SELECT
        branch_id,
        COUNT(*) AS employee_count
    FROM Employees
    GROUP BY branch_id
)
SELECT
    b.branch_name,
    bec.employee_count
FROM BranchEmployeeCount bec
INNER JOIN Branches b
    ON bec.branch_id = b.branch_id
WHERE bec.employee_count > 1;
```

OUTPUT · 3 ROWS

branch_name	employee_count
Main Branch	2
Andheri Branch	2
Bandra Branch	2

Q5 Find Customers with Total Loan Amount Greater Than 1,000,000

```sql
WITH CustomerLoanTotal AS (
    SELECT
        customer_id,
        SUM(loan_amount) AS total_loan_amount
    FROM Loans
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    clt.total_loan_amount
FROM CustomerLoanTotal clt
INNER JOIN Customers c
    ON clt.customer_id = c.customer_id
WHERE clt.total_loan_amount > 1000000;
```

OUTPUT · 5 ROWS

customer_id	customer_name	total_loan_amount
1	Aarav Sharma	2500000.00
3	Rohan Verma	5000000.00
5	Aditya Mehta	1200000.00
7	Kabir Joshi	3500000.00
9	Vivek Rao	4000000.00

Q6 Find Branches with Average Employee Salary Greater Than 60,000

```sql
WITH BranchSalary AS (
    SELECT
        branch_id,
        AVG(salary) AS average_salary
    FROM Employees
    GROUP BY branch_id
)
SELECT
    b.branch_name,
    ROUND(bs.average_salary, 2) AS average_salary
FROM BranchSalary bs
INNER JOIN Branches b
    ON bs.branch_id = b.branch_id
WHERE bs.average_salary > 60000;
```

OUTPUT · 5 ROWS

branch_name	average_salary
Main Branch	65000.000000
Andheri Branch	71000.000000
Bandra Branch	64000.000000
Pune Branch	78000.000000
Delhi Branch	62000.000000

Q7 Find Customers with More Than One Account

```sql
WITH CustomerAccountCount AS (
    SELECT
        customer_id,
        COUNT(*) AS account_count
    FROM Accounts
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    cac.account_count
FROM CustomerAccountCount cac
INNER JOIN Customers c
    ON cac.customer_id = c.customer_id
WHERE cac.account_count > 1;
```

OUTPUT · 0 ROWS

customer_id	customer_name	account_count

Empty set: no rows match this condition in the sample data.

Q8 Find Loan Types with Average Interest Rate Above 9%

```sql
WITH LoanInterestSummary AS (
    SELECT
        loan_type,
        AVG(interest_rate) AS average_interest_rate
    FROM Loans
    GROUP BY loan_type
)
SELECT
    loan_type,
    ROUND(average_interest_rate, 2) AS average_interest_rate
FROM LoanInterestSummary
WHERE average_interest_rate > 9;
```

OUTPUT · 2 ROWS

loan_type	average_interest_rate
Business Loan	9.380000
Personal Loan	10.750000

51 Set Operations

PART III

Q1 Combine Customer and Employee Names

```sql
SELECT
    CONCAT(first_name, ' ', last_name) AS person_name
FROM Customers

UNION

SELECT
    CONCAT(first_name, ' ', last_name) AS person_name
FROM Employees;
```

OUTPUT · 19 ROWS

person_name
Aarav Sharma
Aditya Mehta
Aman Khan
Ananya Patel
Arjun Mehta
Diya Kapoor
Isha Gupta
Kabir Joshi
Karan Rao
Kavita Joshi
Meera Singh
Nisha Singh
Pooja Gupta
Rahul Sharma
Rohan Verma
Rohit Verma
Simran Khan
Sneha Patel
Vivek Rao

Q2 Combine All Customer and Employee Names

```sql
SELECT
    CONCAT(first_name, ' ', last_name) AS person_name
FROM Customers

UNION ALL

SELECT
    CONCAT(first_name, ' ', last_name) AS person_name
FROM Employees;
```

OUTPUT · 19 ROWS

person_name
Aarav Sharma
Ananya Patel
Rohan Verma
Simran Khan
Aditya Mehta
Isha Gupta
Kabir Joshi
Meera Singh
Vivek Rao
Diya Kapoor
Rahul Sharma
Sneha Patel
Aman Khan
Pooja Gupta
Rohit Verma
Kavita Joshi
Arjun Mehta
Nisha Singh
Karan Rao

Q3 Combine Account and Loan Customer IDs

```sql
SELECT
    customer_id
FROM Accounts

UNION

SELECT
    customer_id
FROM Loans;
```

OUTPUT · 10 ROWS

customer_id
1
2
3
4

customer_id
5
6
7
8
9
10

Q4 Combine Customer IDs from Accounts and Loans Without Removing Duplicates

```sql
SELECT
    customer_id
FROM Accounts

UNION ALL

SELECT
    customer_id
FROM Loans;
```

OUTPUT · 20 ROWS

customer_id
1
2
3
4
5
6
7
8
9
10
1
2
3
4
5
6
7
8
9
10

Q5 Find Common Customer IDs Using INTERSECT

```
SELECT
    customer_id
FROM Accounts

INTERSECT

SELECT
    customer_id
FROM Loans;
```

OUTPUT · 10 ROWS

customer_id
1
2
3
4
5
6
7
8
9
10

Q6 Find Customer IDs Present in Transactions or Fund Transfers

```
SELECT customer_id
FROM Customers
WHERE customer_id IN (
    SELECT a.customer_id
    FROM Accounts a
    INNER JOIN Transactions t
        ON a.account_id = t.account_id

    UNION

    SELECT a.customer_id
    FROM Accounts a
    INNER JOIN FundTransfers ft
        ON a.account_id = ft.sender_account_id
);
```

OUTPUT · 10 ROWS

customer_id
1
2
3
4
5
6
7
8
9
10

Q7 Find Customer IDs Who Have Loans but No Fund Transfers

```sql
SELECT customer_id
FROM Loans

EXCEPT

SELECT a.customer_id
FROM Accounts a
INNER JOIN FundTransfers ft
    ON a.account_id = ft.sender_account_id;
```

OUTPUT · 0 ROWS

customer_id

Empty set: no rows match this condition in the sample data.

Q8 Find Account IDs That Have No Transactions

```sql
SELECT account_id
FROM Accounts
WHERE account_id NOT IN (
    SELECT account_id
    FROM Transactions
);
```

OUTPUT · 0 ROWS

account_id

Empty set: no rows match this condition in the sample data.

52 Window Functions

PART III

Q1 Assign a Row Number to Each Account Based on Balance

```sql
SELECT
    account_id,
    account_number,
    balance,
    ROW_NUMBER() OVER (ORDER BY balance DESC) AS rw_number
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	balance	rw_number
9	ACC100009	200000.00	1
3	ACC100003	150000.00	2
7	ACC100007	120000.00	3
10	ACC100010	95000.00	4
5	ACC100005	85000.00	5
1	ACC100001	75000.00	6
4	ACC100004	60000.00	7
6	ACC100006	55000.00	8
2	ACC100002	45000.00	9
8	ACC100008	35000.00	10

Q2 Rank Accounts by Balance Within Each Branch

```sql
SELECT
    account_id,
    account_number,
    branch_id,
    balance,
    RANK() OVER (
        PARTITION BY branch_id
        ORDER BY balance DESC
    ) AS balance_rank
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	branch_id	balance	balance_rank
7	ACC100007	1	120000.00	1
1	ACC100001	1	75000.00	2
2	ACC100002	2	45000.00	1
3	ACC100003	3	150000.00	1
4	ACC100004	4	60000.00	1
8	ACC100008	4	35000.00	2
10	ACC100010	5	95000.00	1
5	ACC100005	5	85000.00	2
9	ACC100009	6	200000.00	1
6	ACC100006	6	55000.00	2

Q3 Assign a Dense Rank to Customers Based on Their Account Balance

```sql
SELECT
    a.account_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.balance,
    DENSE_RANK() OVER (
        ORDER BY a.balance DESC
    ) AS balance_rank
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id;
```

OUTPUT · 10 ROWS

account_id	customer_name	balance	balance_rank
9	Vivek Rao	200000.00	1
3	Rohan Verma	150000.00	2
7	Kabir Joshi	120000.00	3
10	Diya Kapoor	95000.00	4
5	Aditya Mehta	85000.00	5
1	Aarav Sharma	75000.00	6
4	Simran Khan	60000.00	7
6	Isha Gupta	55000.00	8
2	Ananya Patel	45000.00	9
8	Meera Singh	35000.00	10

Q4 Calculate the Running Total of Account Balances

```sql
SELECT
    account_id,
    account_number,
    balance,
    SUM(balance) OVER (
        ORDER BY account_id
    ) AS running_total
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	balance	running_total
1	ACC100001	75000.00	75000.00
2	ACC100002	45000.00	120000.00
3	ACC100003	150000.00	270000.00
4	ACC100004	60000.00	330000.00
5	ACC100005	85000.00	415000.00
6	ACC100006	55000.00	470000.00
7	ACC100007	120000.00	590000.00
8	ACC100008	35000.00	625000.00
9	ACC100009	200000.00	825000.00
10	ACC100010	95000.00	920000.00

Q5 Show the Previous Account Balance Using LAG

```sql
SELECT
    account_id,
    account_number,
    balance,
    LAG(balance) OVER (
        ORDER BY account_id
    ) AS previous_balance
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	balance	previous_balance
1	ACC100001	75000.00	*NULL*
2	ACC100002	45000.00	75000
3	ACC100003	150000.00	45000
4	ACC100004	60000.00	150000
5	ACC100005	85000.00	60000
6	ACC100006	55000.00	85000
7	ACC100007	120000.00	55000
8	ACC100008	35000.00	120000
9	ACC100009	200000.00	35000
10	ACC100010	95000.00	200000

Q6 Show the Next Account Balance Using LEAD

```sql
SELECT
    account_id,
    account_number,
    balance,
    LEAD(balance) OVER (
        ORDER BY account_id
    ) AS next_balance
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	balance	next_balance
1	ACC100001	75000.00	45000
2	ACC100002	45000.00	150000
3	ACC100003	150000.00	60000
4	ACC100004	60000.00	85000
5	ACC100005	85000.00	55000
6	ACC100006	55000.00	120000
7	ACC100007	120000.00	35000
8	ACC100008	35000.00	200000
9	ACC100009	200000.00	95000
10	ACC100010	95000.00	*NULL*

Q7 Calculate the Average Account Balance Across All Accounts

```
SELECT
    account_id,
    account_number,
    balance,
    ROUND(
        AVG(balance) OVER (),
        2
    ) AS average_balance
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	balance	average_balance
1	ACC100001	75000.00	92000.00
2	ACC100002	45000.00	92000.00
3	ACC100003	150000.00	92000.00
4	ACC100004	60000.00	92000.00
5	ACC100005	85000.00	92000.00
6	ACC100006	55000.00	92000.00
7	ACC100007	120000.00	92000.00
8	ACC100008	35000.00	92000.00
9	ACC100009	200000.00	92000.00
10	ACC100010	95000.00	92000.00

Q8 Find the Highest Account Balance in Each Branch

```
SELECT
    account_id,
    account_number,
    branch_id,
    balance,
    MAX(balance) OVER (
        PARTITION BY branch_id
    ) AS highest_branch_balance
FROM Accounts;
```

OUTPUT · 10 ROWS

account_id	account_number	branch_id	balance	highest_branch_balance
1	ACC100001	1	75000.00	120000.00
7	ACC100007	1	120000.00	120000.00
2	ACC100002	2	45000.00	45000.00
3	ACC100003	3	150000.00	150000.00
4	ACC100004	4	60000.00	60000.00
8	ACC100008	4	35000.00	60000.00
5	ACC100005	5	85000.00	95000.00
10	ACC100010	5	95000.00	95000.00
6	ACC100006	6	55000.00	200000.00
9	ACC100009	6	200000.00	200000.00

PART IV

Advanced SQL Objects

Views, indexes, stored procedures and functions, triggers and recursive CTEs.

Views
Reusable reports

Routines
Procedures and functions

Triggers
Audit automation

53 Views

PART IV

Q1 Create a View Showing Customer Account Details

```
CREATE VIEW CustomerAccountDetails AS
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    a.balance,
    a.status
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id;
```

SQL Display the view

```
SELECT *
FROM CustomerAccountDetails;
```

OUTPUT · 10 ROWS

customer_id	customer_name	account_number	balance	status
1	Aarav Sharma	ACC100001	75000.00	Active
2	Ananya Patel	ACC100002	45000.00	Active
3	Rohan Verma	ACC100003	150000.00	Active
4	Simran Khan	ACC100004	60000.00	Active
5	Aditya Mehta	ACC100005	85000.00	Active
6	Isha Gupta	ACC100006	55000.00	Active
7	Kabir Joshi	ACC100007	120000.00	Active
8	Meera Singh	ACC100008	35000.00	Active
9	Vivek Rao	ACC100009	200000.00	Active
10	Diya Kapoor	ACC100010	95000.00	Active

Q2 Create a View Showing Customer Loan Details and Display It

```sql
CREATE VIEW CustomerLoanDetails AS
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    l.loan_type,
    l.loan_amount,
    l.interest_rate,
    l.status
FROM Customers c
INNER JOIN Loans l
    ON c.customer_id = l.customer_id;
```

SQL Display the view

```sql
SELECT *
FROM CustomerLoanDetails;
```

OUTPUT · 10 ROWS

customer_id	customer_name	loan_type	loan_amount	interest_rate	status
1	Aarav Sharma	Home Loan	2500000.00	8.50	Active
2	Ananya Patel	Personal Loan	500000.00	11.00	Active
3	Rohan Verma	Business Loan	5000000.00	9.25	Active
4	Simran Khan	Education Loan	800000.00	7.50	Active
5	Aditya Mehta	Car Loan	1200000.00	9.00	Active
6	Isha Gupta	Personal Loan	300000.00	10.50	Active
7	Kabir Joshi	Home Loan	3500000.00	8.25	Active
8	Meera Singh	Education Loan	600000.00	7.25	Active
9	Vivek Rao	Business Loan	4000000.00	9.50	Active
10	Diya Kapoor	Car Loan	900000.00	8.75	Active

Q3 Create a View Showing Branch Employee Details and Display It

```sql
CREATE VIEW BranchEmployeeDetails AS
SELECT
    b.branch_id,
    b.branch_name,
    e.employee_id,
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    e.job_title,
    e.salary,
    e.status
FROM Branches b
INNER JOIN Employees e
    ON b.branch_id = e.branch_id;
```

SQL **Display the view**

```sql
SELECT *
FROM BranchEmployeeDetails;
```

OUTPUT · 9 ROWS

branch_id	branch_name	employee_id	employee_name	job_title	salary	status
1	Main Branch	1	Rahul Sharma	Branch Manager	85000.00	Active
1	Main Branch	2	Sneha Patel	Cashier	45000.00	Active
2	Andheri Branch	3	Aman Khan	Branch Manager	82000.00	Active
2	Andheri Branch	4	Pooja Gupta	Loan Officer	60000.00	Active
3	Bandra Branch	5	Rohit Verma	Branch Manager	80000.00	Active
3	Bandra Branch	6	Kavita Joshi	Customer Service	48000.00	Active
4	Pune Branch	7	Arjun Mehta	Branch Manager	78000.00	Active
5	Delhi Branch	8	Nisha Singh	Loan Officer	62000.00	Active
6	Bangalore Branch	9	Karan Rao	Cashier	46000.00	Active

Q4 **Create a View Showing Customer Transaction Details and Display It**

```sql
CREATE VIEW CustomerTransactionDetails AS
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.account_number,
    t.transaction_type,
    t.amount,
    t.transaction_date,
    t.status
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN Transactions t
    ON a.account_id = t.account_id;
```

SQL **Display the view**

```sql
SELECT *
FROM CustomerTransactionDetails;
```

OUTPUT · 13 ROWS

customer_id	customer_name	account_number	transaction_type	amount	transaction_date	status
1	Aarav Sharma	ACC100001	Deposit	25000.00	2026-10-29 07:09:08	Success
1	Aarav Sharma	ACC100001	Withdrawal	5000.00	2026-10-29 07:09:08	Success
2	Ananya Patel	ACC100002	Deposit	15000.00	2026-10-29 07:09:08	Success
2	Ananya Patel	ACC100002	Withdrawal	3000.00	2026-10-29 07:09:08	Success
3	Rohan Verma	ACC100003	Deposit	50000.00	2026-10-29 07:09:08	Success
3	Rohan Verma	ACC100003	Transfer	10000.00	2026-10-29 07:09:08	Success
4	Simran Khan	ACC100004	Deposit	20000.00	2026-10-29 07:09:08	Success
5	Aditya Mehta	ACC100005	Deposit	30000.00	2026-10-29 07:09:08	Success
6	Isha Gupta	ACC100006	Withdrawal	7000.00	2026-10-29 07:09:08	Success
7	Kabir Joshi	ACC100007	Deposit	40000.00	2026-10-29 07:09:08	Success
8	Meera Singh	ACC100008	Withdrawal	5000.00	2026-10-29 07:09:08	Success
9	Vivek Rao	ACC100009	Deposit	75000.00	2026-10-29 07:09:08	Success

customer_id	customer_name	account_number	transaction_type	amount	transaction_date	status
10	Diya Kapoor	ACC100010	Transfer	15000.00	2026-10-29 07:09:08	Success

Q5 Create a View Showing Active Accounts with Balance Above ₹50,000

```sql
CREATE VIEW HighBalanceAccounts AS
SELECT
    account_id,
    account_number,
    customer_id,
    balance,
    status
FROM Accounts
WHERE balance > 50000
  AND status = 'Active';
```

SQL Display the view

```sql
SELECT *
FROM HighBalanceAccounts;
```

OUTPUT · 8 ROWS

account_id	account_number	customer_id	balance	status
1	ACC100001	1	75000.00	Active
3	ACC100003	3	150000.00	Active
4	ACC100004	4	60000.00	Active
5	ACC100005	5	85000.00	Active
6	ACC100006	6	55000.00	Active
7	ACC100007	7	120000.00	Active
9	ACC100009	9	200000.00	Active
10	ACC100010	10	95000.00	Active

Q6 Create a View Showing Pending KYC Records and Display It

```sql
CREATE VIEW PendingKYC AS
SELECT
    k.kyc_id,
    k.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    k.document_type,
    k.document_number,
    k.verification_status
FROM KYC k
INNER JOIN Customers c
    ON k.customer_id = c.customer_id
WHERE k.verification_status = 'Pending';
```

SQL **Display the view**

```sql
SELECT *
FROM PendingKYC;
```

OUTPUT · 2 ROWS

kyc_id	customer_id	customer_name	document_type	document_number	verification_status
7	7	Kabir Joshi	PAN	PAN100007	Pending
10	10	Diya Kapoor	PAN	PAN100010	Pending

Q7 **Create a View Showing Customers with Active Loans Above ₹1,000,000**

```sql
CREATE VIEW HighValueLoans AS
SELECT
    l.loan_id,
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    l.loan_type,
    l.loan_amount,
    l.interest_rate,
    l.status
FROM Loans l
INNER JOIN Customers c
    ON l.customer_id = c.customer_id
WHERE l.loan_amount > 1000000
  AND l.status = 'Active';
```

SQL **Display the view**

```sql
SELECT *
FROM HighValueLoans;
```

OUTPUT · 5 ROWS

loan_id	customer_id	customer_name	loan_type	loan_amount	interest_rate	status
1	1	Aarav Sharma	Home Loan	2500000.00	8.50	Active
3	3	Rohan Verma	Business Loan	5000000.00	9.25	Active
5	5	Aditya Mehta	Car Loan	1200000.00	9.00	Active
7	7	Kabir Joshi	Home Loan	3500000.00	8.25	Active
9	9	Vivek Rao	Business Loan	4000000.00	9.50	Active

Q8 **Create a View Showing Account Summary with Customer, Branch, and Account Type**

```sql
CREATE VIEW AccountSummary AS
SELECT
    a.account_id,
    a.account_number,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    b.branch_name,
    at.account_type_name,
    a.balance,
    a.status
FROM Accounts a
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
INNER JOIN Branches b
    ON a.branch_id = b.branch_id
INNER JOIN AccountTypes at
    ON a.account_type_id = at.account_type_id;
```

SQL **Display the view**

```
SELECT *
FROM AccountSummary;
```

OUTPUT · 10 ROWS

account_id	account_number	customer_name	branch_name	account_type_name	balance	status
1	ACC100001	Aarav Sharma	Main Branch	Savings	75000.00	Active
2	ACC100002	Ananya Patel	Andheri Branch	Savings	45000.00	Active
3	ACC100003	Rohan Verma	Bandra Branch	Current	150000.00	Active
4	ACC100004	Simran Khan	Pune Branch	Savings	60000.00	Active
5	ACC100005	Aditya Mehta	Delhi Branch	Salary	85000.00	Active
6	ACC100006	Isha Gupta	Bangalore Branch	Savings	55000.00	Active
7	ACC100007	Kabir Joshi	Main Branch	Senior Citizen Savings	120000.00	Active
8	ACC100008	Meera Singh	Pune Branch	Savings	35000.00	Active
9	ACC100009	Vivek Rao	Bangalore Branch	Current	200000.00	Active
10	ACC100010	Diya Kapoor	Delhi Branch	NRI Savings	95000.00	Active

54 Indexes

PART IV

Q1 **Create an Index on Customer Email**

```
CREATE INDEX idx_customer_email
ON Customers(email);
```

SQL **Indexes**

```
SHOW INDEX FROM Customers;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q2 **Create and Verify an Index on Account Balance**

```
CREATE INDEX idx_account_balance
ON Accounts(balance);
```

SQL **Indexes**

```
SHOW INDEX FROM Accounts;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q3 **Create and Verify an Index on Transaction Account ID**

```
CREATE INDEX idx_transaction_account
ON Transactions(account_id);
```

SQL **Indexes**

```
SHOW INDEX FROM Transactions;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q4 **Create and Verify a Composite Index on Customer and Status**

```
CREATE INDEX idx_customer_status
ON Accounts(customer_id, status);
```

SQL **Indexes**

```
SHOW INDEX FROM Accounts;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q5 **Create and Verify an Index on Loan Customer ID**

```
CREATE INDEX idx_loan_customer
ON Loans(customer_id);
```

SQL **Indexes**

```
SHOW INDEX FROM Loans;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q6 Create and Verify an Index on Fund Transfer Date

```
CREATE INDEX idx_transfer_date
ON FundTransfers(transfer_date);
```

SQL Indexes

```
SHOW INDEX FROM FundTransfers;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q7 Create and Verify an Index on Customer Phone

```
CREATE INDEX idx_customer_phone
ON Customers(phone);
```

SQL Indexes

```
SHOW INDEX FROM Customers;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

Q8 Create and Verify a Composite Index on Loan Type and Status

```
CREATE INDEX idx_loan_type_status
ON Loans(loan_type, status);
```

SQL Indexes

```
SHOW INDEX FROM Loans;
```

SHOW INDEX returns MySQL server index metadata and is not tabulated here.

55 Stored Procedures PART IV

Definition step: stored routines, triggers and index metadata are MySQL server features and have no tabular output to display here. Run the script in MySQL Workbench to execute them.

Q1 Create a Stored Procedure to Display All Customers

```
DELIMITER //

CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT
        customer_id,
        CONCAT(first_name, ' ', last_name) AS customer_name,
        email,
        phone,
        city,
        state,
        status
    FROM Customers;
END //

DELIMITER ;

CALL GetAllCustomers();
```

Q2 Create a Stored Procedure to Find Customers by City

```
DELIMITER //

CREATE PROCEDURE GetCustomersByCity(IN city_name VARCHAR(50))
BEGIN
    SELECT
        customer_id,
        CONCAT(first_name, ' ', last_name) AS customer_name,
        email,
        phone,
        city,
        state
    FROM Customers
    WHERE city = city_name;
END //

DELIMITER ;
```

Q3 Create a Stored Procedure to Find Accounts Above a Given Balance

```
DELIMITER //

CREATE PROCEDURE GetAccountsAboveBalance(IN min_balance DECIMAL(15,2))
BEGIN
    SELECT
        account_id,
        account_number,
        customer_id,
        balance,
        status
    FROM Accounts
    WHERE balance > min_balance;
END //

DELIMITER ;
```

Q4 Create a Stored Procedure to Find Loans by Loan Type

```
DELIMITER //

CREATE PROCEDURE GetLoansByType(IN p_loan_type VARCHAR(50))
BEGIN
    SELECT
        loan_id,
        customer_id,
        loan_type,
        loan_amount,
        interest_rate,
        tenure_months,
        status
    FROM Loans
    WHERE Loans.loan_type = p_loan_type;
END //

DELIMITER ;
```

Q5 Create a Stored Procedure to Find Transactions by Account

```
DELIMITER //

CREATE PROCEDURE GetTransactionsByAccount(IN acc_id INT)
BEGIN
    SELECT
        transaction_id,
        account_id,
        transaction_type,
        amount,
        transaction_date,
        description,
        status
    FROM Transactions
    WHERE account_id = acc_id;
END //

DELIMITER ;
```

Q6 Create a Stored Procedure to Find Employees by Branch

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByBranch(IN br_id INT)
BEGIN
    SELECT
        employee_id,
        branch_id,
        CONCAT(first_name, ' ', last_name) AS employee_name,
        job_title,
        salary,
        status
    FROM Employees
    WHERE branch_id = br_id;
END //

DELIMITER ;
```

Q7 Create a Stored Procedure to Find Customers with Pending KYC

```
DELIMITER //

CREATE PROCEDURE GetPendingKYCCustomers()
BEGIN
    SELECT
        c.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
        k.document_type,
        k.document_number,
        k.verification_status
    FROM Customers c
    INNER JOIN KYC k
        ON c.customer_id = k.customer_id
    WHERE k.verification_status = 'Pending';
END //

DELIMITER ;
```

Q8 Create a Stored Procedure to Find High-Value Loans

```
DELIMITER //

CREATE PROCEDURE GetHighValueLoans(IN min_amount DECIMAL(15,2))
BEGIN
    SELECT
        l.loan_id,
        l.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
        l.loan_type,
        l.loan_amount,
        l.interest_rate,
        l.status
    FROM Loans l
    INNER JOIN Customers c
        ON l.customer_id = c.customer_id
    WHERE l.loan_amount > min_amount;

END //
DELIMITER ;
```

56 Stored Procedure Execution — PART IV

Procedure calls: each CALL returns the result set of its stored procedure, which needs a MySQL server to run.

Q1 Execute the Procedure to Display All Customers

```
CALL GetAllCustomers();
```

Q2 Execute the Procedure to Find Customers by City

```
CALL GetCustomersByCity('Mumbai');
```

Q3 Execute the Procedure to Find Accounts Above ₹100,000

```
CALL GetAccountsAboveBalance(100000);
```

Q4 Execute the Procedure to Find Home Loans

```
CALL GetLoansByType('Home Loan');
```

Q5 Execute the Procedure to Find Transactions for Account 1

```
CALL GetTransactionsByAccount(1);
```

Q6 Execute the Procedure to Find Employees in Branch 1

```
CALL GetEmployeesByBranch(1);
```

Q7 Execute the Procedure to Find Customers with Pending KYC

```
CALL GetPendingKYCCustomers();
```

Q8 Execute the Procedure to Find Loans Above ₹1,000,000

```
CALL GetHighValueLoans(1000000);
```

57 Stored Functions

PART IV

Definition step: stored routines, triggers and index metadata are MySQL server features and have no tabular output to display here. Run the script in MySQL Workbench to execute them.

Q1 Create a function that returns the full name of a customer using customer ID.

```
DELIMITER //

CREATE FUNCTION GetCustomerName(p_customer_id INT)
RETURNS VARCHAR(101)
DETERMINISTIC
BEGIN
    DECLARE full_name VARCHAR(101);

    SELECT CONCAT(first_name, ' ', last_name)
    INTO full_name
    FROM Customers
    WHERE customer_id = p_customer_id;

    RETURN full_name;
END //

DELIMITER ;
SELECT GetCustomerName(1) AS customer_name;
```

Q2 Create a function that returns the balance of an account using account ID.

```
DELIMITER //

CREATE FUNCTION GetAccountBalance(p_account_id INT)
RETURNS DECIMAL(15,2)
DETERMINISTIC
BEGIN
    DECLARE account_balance DECIMAL(15,2);

    SELECT balance
    INTO account_balance
    FROM Accounts
    WHERE account_id = p_account_id;

    RETURN account_balance;
END //

DELIMITER ;
SELECT GetAccountBalance(1) AS account_balance;
```

Q3 Create a function that returns the total loan amount of a customer using customer ID.

```
DELIMITER //

CREATE FUNCTION GetCustomerLoanAmount(p_customer_id INT)
RETURNS DECIMAL(15,2)
DETERMINISTIC
BEGIN
    DECLARE total_loan DECIMAL(15,2);

    SELECT COALESCE(SUM(loan_amount), 0)
    INTO total_loan
    FROM Loans
    WHERE customer_id = p_customer_id;

    RETURN total_loan;
END //

DELIMITER ;
SELECT GetCustomerLoanAmount(1) AS total_loan_amount;
```

Q4 Create a function that returns the total number of accounts owned by a customer.

```
DELIMITER //

CREATE FUNCTION GetCustomerAccountCount(p_customer_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE account_count INT;

    SELECT COUNT(*)
    INTO account_count
    FROM Accounts
    WHERE customer_id = p_customer_id;

    RETURN account_count;
END //

DELIMITER ;
SELECT GetCustomerAccountCount(1) AS account_count;
```

Q5 Create a function that returns the total transaction amount for an account.

```
DELIMITER //

CREATE FUNCTION GetTotalTransactionAmount(p_account_id INT)
RETURNS DECIMAL(15,2)
DETERMINISTIC
BEGIN
    DECLARE total_amount DECIMAL(15,2);

    SELECT COALESCE(SUM(amount), 0)
    INTO total_amount
    FROM Transactions
    WHERE account_id = p_account_id;

    RETURN total_amount;
END //

DELIMITER ;
SELECT GetTotalTransactionAmount(1) AS total_transaction_amount;
```

Q6 Create a function that returns the average interest rate of a given loan type.

```
DELIMITER //

CREATE FUNCTION GetAverageLoanInterestRate(p_loan_type VARCHAR(50))
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE avg_rate DECIMAL(5,2);

    SELECT COALESCE(AVG(interest_rate), 0)
    INTO avg_rate
    FROM Loans
    WHERE loan_type = p_loan_type;

    RETURN avg_rate;
END //

DELIMITER ;
SELECT GetAverageLoanInterestRate('Home Loan') AS average_interest_rate;
```

Q7 Create a function that returns the number of pending KYC records.

```
DELIMITER //

CREATE FUNCTION GetPendingKYCCount()
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE pending_count INT;

    SELECT COUNT(*)
    INTO pending_count
    FROM KYC
    WHERE verification_status = 'Pending';

    RETURN pending_count;
END //

DELIMITER ;
SELECT GetPendingKYCCount() AS pending_kyc_count;
```

Q8 **Create a function that returns the total loan payments made for a given loan.**

```
DELIMITER //

CREATE FUNCTION GetTotalLoanPayments(p_loan_id INT)
RETURNS DECIMAL(15,2)
DETERMINISTIC
BEGIN
    DECLARE total_payment DECIMAL(15,2);

    SELECT COALESCE(SUM(payment_amount), 0)
    INTO total_payment
    FROM LoanPayments
    WHERE loan_id = p_loan_id;

    RETURN total_payment;
END //

DELIMITER ;
SELECT GetTotalLoanPayments(1) AS total_loan_payments;
```

58 Triggers

PART IV

Definition step: stored routines, triggers and index metadata are MySQL server features and have no tabular output to display here. Run the script in MySQL Workbench to execute them.

Q1 **Create a trigger that automatically records a new transaction in the AuditLogs table whenever a transaction is inserted.**

```
DELIMITER //

CREATE TRIGGER AfterTransactionInsert
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs
    (action_type, table_name, record_id, action_description)
    VALUES
    (
        'INSERT',
        'Transactions',
        NEW.transaction_id,
        CONCAT(
            'New transaction inserted. Type: ',
            NEW.transaction_type,
            ', Amount: ',
            NEW.amount
        )
    );
END //

DELIMITER ;

-- Insert a test transaction

INSERT INTO Transactions
(account_id, transaction_type, amount, description, reference_number)
VALUES
(1, 'Deposit', 1000, 'Trigger Test Transaction', 'TXNTEST001');

-- Check the automatically created audit log

SELECT *
FROM AuditLogs WHERE table_name = 'Transactions'
ORDER BY log_id DESC
LIMIT 1;
```

Q2 **Create a trigger that automatically records an audit log whenever an account balance is updated.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterAccountBalanceUpdate
AFTER UPDATE ON Accounts
FOR EACH ROW
BEGIN
    IF OLD.balance <> NEW.balance THEN
        INSERT INTO AuditLogs
        (action_type, table_name, record_id, action_description)
        VALUES
        (
            'UPDATE',
            'Accounts',
            NEW.account_id,
            CONCAT(
                'Account balance changed from ',
                OLD.balance,
                ' to ',
                NEW.balance
            )
        );
    END IF;
END //

DELIMITER ;

-- Update an account balance

UPDATE Accounts
SET balance = balance + 1000
WHERE account_id = 1;

-- Check the automatically created audit log
SELECT *
FROM AuditLogs
WHERE table_name = 'Accounts'
ORDER BY log_id DESC
LIMIT 1;
```

Q3 **Create a trigger that automatically records an audit log whenever a new customer is inserted.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterCustomerInsert
AFTER INSERT ON Customers
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs
    (customer_id, action_type, table_name, record_id, action_description)
    VALUES
    (
        NEW.customer_id,
        'INSERT',
        'Customers',
        NEW.customer_id,
        CONCAT(
            'New customer registered: ',
```

```
            NEW.first_name,
            ' ',
            NEW.last_name
        )
    );
END //

DELIMITER ;

-- Insert a test customer

INSERT INTO Customers
(first_name, last_name, email, phone, city, state, pincode)
VALUES
('Test', 'Customer', 'test.customer@example.com', '9999999999',
 'Mumbai', 'Maharashtra', '400001');

-- Check the automatically created audit log
SELECT *
FROM AuditLogs
WHERE table_name = 'Customers'
ORDER BY log_id DESC
LIMIT 1;
```

Q4 **Create a trigger that automatically records an audit log whenever a loan payment is inserted.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterLoanPaymentInsert
AFTER INSERT ON LoanPayments
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs
    (action_type, table_name, record_id, action_description)
    VALUES
    (
        'INSERT',
        'LoanPayments',
        NEW.payment_id,
        CONCAT(
            'New loan payment inserted. Loan ID: ',
            NEW.loan_id,
            ', Amount: ',
            NEW.payment_amount
        )
    );
END //

DELIMITER ;

-- Insert a test loan payment

INSERT INTO LoanPayments
(loan_id, payment_amount, payment_method, transaction_reference, remarks)
VALUES
(1, 10000, 'Online', 'LPTEST001', 'Trigger Test Payment');

-- Check the automatically created audit log

SELECT *
FROM AuditLogs
WHERE table_name = 'LoanPayments'
ORDER BY log_id DESC
LIMIT 1;
```

Q5 **Create a trigger that automatically records an audit log whenever a complaint status is updated.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterComplaintStatusUpdate
AFTER UPDATE ON Complaints
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        INSERT INTO AuditLogs
        (customer_id, employee_id, action_type, table_name, record_id, action_description)
        VALUES
        (
            NEW.customer_id,
            NEW.assigned_employee_id,
            'UPDATE',
            'Complaints',
            NEW.complaint_id,
            CONCAT(
                'Complaint status changed from ',
                OLD.status,
                ' to ',
                NEW.status
            )
        );
    END IF;
END //

DELIMITER ;

-- Update a complaint status

UPDATE Complaints
SET status = 'In Progress'
WHERE complaint_id = 1;
-- Check the automatically created audit log

SELECT *
FROM AuditLogs
WHERE table_name = 'Complaints'
ORDER BY log_id DESC
LIMIT 1;
```

Q6 **Create a trigger that automatically records an audit log whenever a new fund transfer is inserted.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterFundTransferInsert
AFTER INSERT ON FundTransfers
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs
    (action_type, table_name, record_id, action_description)
    VALUES
    (
        'INSERT',
        'FundTransfers',
        NEW.transfer_id,
        CONCAT(
            'New fund transfer inserted. Amount: ',
            NEW.amount,
```

```
            ', Type: ',
            NEW.transfer_type
        )
    );
END //

DELIMITER ;

-- Insert a test fund transfer

INSERT INTO FundTransfers
(sender_account_id, receiver_account_id, amount, transfer_type, reference_number, remarks)
VALUES
(1, 2, 2000, 'NEFT', 'FTTEST001', 'Trigger Test Transfer');

-- Check the automatically created audit log

SELECT *
FROM AuditLogs
WHERE table_name = 'FundTransfers'
ORDER BY log_id DESC
LIMIT 1;
```

Q7 **Create a trigger that automatically records an audit log whenever a KYC status is updated.**

```
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterKYCStatusUpdate
AFTER UPDATE ON KYC
FOR EACH ROW
BEGIN
    IF OLD.verification_status <> NEW.verification_status THEN
        INSERT INTO AuditLogs
        (customer_id, action_type, table_name, record_id, action_description)
        VALUES
        (
            NEW.customer_id,
            'UPDATE',
            'KYC',
            NEW.kyc_id,
            CONCAT(
                'KYC status changed from ',
                OLD.verification_status,
                ' to ',
                NEW.verification_status
            )
        );
    END IF;
END //

DELIMITER ;

-- Update a KYC status

UPDATE KYC
SET verification_status = 'Verified'
WHERE kyc_id = 7;

-- Check the automatically created audit log
SELECT *
FROM AuditLogs
WHERE table_name = 'KYC'
ORDER BY log_id DESC
LIMIT 1;
```

Q8 **Create a trigger that automatically records an audit log whenever a loan record is deleted.**

```sql
-- Create the trigger

DELIMITER //

CREATE TRIGGER AfterLoanDelete
AFTER DELETE ON Loans
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs
    (customer_id, action_type, table_name, record_id, action_description)
    VALUES
    (
        OLD.customer_id,
        'DELETE',
        'Loans',
        OLD.loan_id,
        CONCAT(
            'Loan deleted. Loan type: ',
            OLD.loan_type,
            ', Loan amount: ',
            OLD.loan_amount
        )
    );
END //

DELIMITER ;

-- Insert a temporary loan for testing

INSERT INTO Loans
(customer_id, branch_id, loan_type, loan_amount, interest_rate, tenure_months, start_date, end_date)
VALUES
(11, 1, 'Test Loan', 10000, 5.00, 12, '2026-01-01', '2027-01-01');

-- Delete the temporary loan
DELETE FROM Loans
WHERE loan_type = 'Test Loan'
  AND customer_id = 11;

-- Check the automatically created audit log

SELECT *
FROM AuditLogs
WHERE table_name = 'Loans'
AND action_type = 'DELETE'
ORDER BY log_id DESC
LIMIT 1;
```

59 Recursive CTE

PART IV

Q1 Create a recursive CTE to display a hierarchy of branches from level 1 to level 3.

```sql
WITH RECURSIVE BranchHierarchy AS (

    -- Anchor member
    SELECT
        branch_id,
        branch_name,
        1 AS level
    FROM Branches
    WHERE branch_id = 1

    UNION ALL

    -- Recursive member
    SELECT
        b.branch_id,
        b.branch_name,
        bh.level + 1
    FROM Branches b
    INNER JOIN BranchHierarchy bh
        ON b.branch_id = bh.branch_id + 1
    WHERE bh.level < 3
)

SELECT
    branch_id,
    branch_name,
    level
FROM BranchHierarchy;
```

OUTPUT · 3 ROWS

branch_id	branch_name	level
1	Main Branch	1
2	Andheri Branch	2
3	Bandra Branch	3

Q2 Create a recursive CTE to generate numbers from 1 to 10.

```sql
WITH RECURSIVE NumberSeries AS (

    -- Starting value
    SELECT 1 AS number

    UNION ALL

    -- Generate next number
    SELECT number + 1
    FROM NumberSeries
    WHERE number < 10
)

SELECT number
FROM NumberSeries;
```

OUTPUT · 10 ROWS

number
1
2
3
4

number
5
6
7
8
9
10

Q3 **Create a recursive CTE to generate monthly periods from January 2026 to June 2026.**

```
WITH RECURSIVE MonthlyPeriods AS (

    SELECT DATE('2026-01-01') AS month_start

    UNION ALL

    SELECT DATE_ADD(month_start, INTERVAL 1 MONTH)
    FROM MonthlyPeriods
    WHERE month_start < '2026-06-01'
)

SELECT
    month_start,
    DATE_FORMAT(month_start, '%M %Y') AS month_name
FROM MonthlyPeriods;
```

OUTPUT · 6 ROWS

month_start	month_name
2026-01-01	January 2026
2026-02-01	February 2026
2026-03-01	March 2026
2026-04-01	April 2026
2026-05-01	May 2026
2026-06-01	June 2026

Q4 **Create a recursive CTE to generate weekly dates from January 1, 2026 to January 31, 2026.**

```
WITH RECURSIVE WeeklyDates AS (

    SELECT DATE('2026-01-01') AS week_date

    UNION ALL

    SELECT DATE_ADD(week_date, INTERVAL 7 DAY)
    FROM WeeklyDates
    WHERE week_date < '2026-01-31'
)

SELECT
    week_date
FROM WeeklyDates;
```

OUTPUT · 6 ROWS

week_date
2026-01-01
2026-01-08
2026-01-15
2026-01-22

week_date
2026-01-29
2026-02-05

Conclusion

The project demonstrates how a normalised relational schema, carefully ordered seed data and a progressive set of SQL techniques can model and analyse real banking operations. It covers the full path from DDL and constraints, through filtering, aggregation, joins, subqueries, CTEs and window functions, to reusable views, indexes, stored routines, audit triggers and recursive queries.

SKILLS SHOWCASED

Database Design Entity modelling, keys, constraints	**Analytical SQL** Joins, subqueries, windows	**Programmability** Views, routines, triggers

> Result tables in this document were generated by executing the project queries on the seed data in a MySQL-compatible emulation; row ordering for queries without ORDER BY and timestamps populated by CURRENT_TIMESTAMP may differ from a live MySQL session.